



Evaluating The Efficacy of Pretrained Models for Semantic Segmentation in the Context of Dietetics

Robert Flanagan (21311625)
BSC Computer Systems (LM051)
Department of Computer Science and Information Systems (CSIS)
University of Limerick

April 28, 2025

This final year project report was completed in compliance with the FYP guidelines
2024-2025, under the supervision of Mr. J.J. Collins.

Abstract

This project investigates the effectiveness of existing semantic segmentation models for food image analysis and proposes a novel dual-branch architecture, Blend-Seg, to address key challenges. While traditional convolutional neural networks (CNNs) like U-Net and DeepLab offer strong pixel-level predictions, they often suffer from over-segmentation and poor object localisation when applied to complex food imagery. Through a series of experiments on the FoodSeg103 dataset, this project benchmarks U-Net++, DeepLabV3+, and YOLOv8 models, identifying their limitations and potential areas for improvement.

To overcome these challenges, the Blend-Seg architecture was developed. Blend-Seg combines the outputs of a pixel-based U-Net++ and an object-detection YOLOv8 model, merging fine-grained and object-level features through a lightweight, learnable fusion module. Experiments demonstrate that Blend-Seg achieves a mean Intersection-over-Union (mIoU) score of 41.6%, outperforming standard CNN architectures while maintaining real-time inference speeds suitable for mobile and dietary applications. Compared to DeepLab with Dense CRF post-processing, Blend-Seg improves mIoU by over 10 percentage points while reducing inference time by a factor of 17.

The findings highlight the value of combining object detection and segmentation in a unified, efficient framework. This work contributes a practical, scalable solution toward more accurate automatic food recognition, with potential impacts for nutrition tracking, health monitoring, and real-time dietary assistance systems.

Declaration

I hereby declare that I have produced this paper without the prohibited assistance of third parties and without making use of aids other than those specified; notions taken over directly or indirectly from other sources have been identified as such. This paper has not previously been presented in identical or similar form to any other Irish or foreign examination board.

Ethics Declaration

I hereby declare that my project does not involve human participants in any way and that I therefore was not required to submit an ethics application.

AI Declaration

I hereby declare that I have not used artificial intelligence to produce my report or project .

Contents

Acknowledgements	vi
1 Introduction	1
1.1 Summary	1
1.2 Research Questions	2
1.3 New Contributions	2
1.4 Methodology	3
1.5 Overview of the Report	4
1.6 Motivation	5
2 Literature Review	6
2.1 Overview of Artificial Intelligence	6
2.2 Artificial Neural Networks	6
2.2.1 Perceptron	7
2.2.2 Multilayer Perceptron	8
2.2.3 Backpropagation	8
2.3 Convolutional Neural Networks	9
2.3.1 Convolutional Layer	9
2.3.2 Activation Function	10
2.3.3 Pooling Layer	11
2.3.4 Fully Connected Layer	12
2.3.5 ResNet	12
2.3.6 You Only Look Once (YOLO)	13
2.4 Semantic Segmentation	14
2.4.1 U-Net	15
2.4.2 DeepLab	16
2.4.3 Mask R-CNN	18
2.4.4 Two Stage Detection + Segmentation Techniques	19
2.4.5 Performance Metrics	20
2.5 Training Data	21
2.5.1 Creating a Training Dataset	21
2.5.2 Food-Seg103	23
3 Prototype	25
3.1 Introduction	25
3.2 Development Environment	25
3.3 Architecture Details	25
3.4 Implementation	27
3.4.1 Class Mappings and Data Loading	27
3.4.2 YOLO Map Generation	27
3.4.3 Blend-Seg Architecture	29

3.4.4	Loss Function	30
3.4.5	U-Net Unfreezing	31
3.5	Evaluation and Results	32
3.5.1	Overall Validation Metrics	32
3.5.2	Visualisation of Results	32
3.6	Conclusion	34
4	Experiments	35
4.1	Overview of the Empirical Framework	35
4.1.1	Dataset and Preparation	36
4.2	Experiment 1: DeepLab	36
4.3	Experiment 2: DeepLab	39
4.4	Experiment 3: DeepLab	42
4.5	Experiment 4: DeepLab	45
4.6	Experiment 5: DeepLab	47
4.7	Experiment 6: DeepLab	49
4.8	Overall Evaluation of DeepLab Experiments	52
4.9	Experiment 7: U-Net	53
4.10	Experiment 8: U-Net	55
4.11	Experiment 9: U-Net	57
4.12	Overall Evaluation of U-Net Experiments	59
4.13	Comparison of U-Net VS DeepLab	59
4.14	Experiment 10: YOLO	60
4.15	Experiment 11: YOLO	63
4.16	Experiment 12: Blend-Seg	65
4.17	Experiment 13: Blend-Seg	67
4.18	Experiment 14: Blend-Seg	69
4.19	Final Evaluation of Experiments	71
4.20	Comparison to Existing Models	72
4.21	Threats to Validity	73
5	Discussion and Conclusion	74

Acknowledgements

I would like to thank my supervisor, Mr J.J. Collins, for the continued guidance and feedback throughout the writing of this report. His knowledge and insight proved invaluable in guiding the technical direction of my research, while his constructive advice helped me critically assess and refine both my ideas and writing style.

I would also like to thank my parents for their constant support and encouragement throughout my studies.

1 Introduction

This section provides a summary of the FYP and the motivations behind it as well as defining the research questions.

1.1 Summary

This final year project aims to compare and contrast Convolutional Neural Networks (CNNs) designed for semantic segmentation in the context of dietary image analysis, with the goal of improving pixel-level segmentation accuracy. These semantic segmentation CNNs work by looking at an image and assigning a label to every pixel. (Mo et al. 2022). For example, when presented with a picture of fish and chips, the network will label all the pixels in the fish region as "fish" and all the pixels in the chips region as "chips", every other pixel is labelled as background. An example of a segmented food image can be seen in Figure 1.

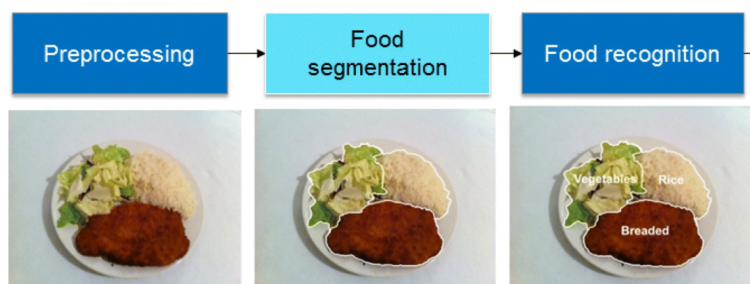


Figure 1: Example of a labelled segmented food image (Aslan et al. 2020)

The CNNs mentioned above are examples of Artificial Neural Networks (ANNs). ANNs are a class of machine learning models inspired by the structure and function of the human brain. They work by processing data through weighted connections and non-linear mathematical operations allowing them to learn complex relationships within the data (Steven Walczak and Cerpa 2006). The first ANNs were called perceptrons, these were simple models consisting of a single layer of neurons (Fradkov 2020). These early models were limited in their capabilities, leading to the development of Multi-Layer Perceptrons (MLPs), which include multiple layers of neurons, allowing for more complex representations of data. (Popescu et al. 2012) One big breakthrough in the field of ANNs was the introduction of the backpropagation learning algorithm which adjusts the weights of the ANN based on the error in the predictions (J. Li et al. 2016). Since then, ANNs have advanced rapidly, leading to more complex and deep networks enabling them to tackle increasingly difficult tasks.

These advancements eventually led to the creation of the CNN. A CNN is a deep learning model primarily used for image analysis. They are comprised of 3 layers these being the convolutional layers which detect features in the image by applying a filter over the input, the pooling layers which reduce data to highlight the key features and the fully connected layers which connect the outputs of all layers to create a final result (O'Shea and Nash 2015)

Early CNNs output a single label per image, dietary analysis requires precise labels and object localisation for each food item and ingredient, hence the need for semantic segmentation. Semantic segmentation assigns every pixel its own class label (Y. Guo et al. 2018). In this report I examine two leading architectures, U-Net and DeepLab and compare their performance on key metrics.

By contrast, object detection CNNs such as You Only Look Once (YOLO) operate by drawing bounding boxes around items and assigning a class label to each box (Redmon et al. 2016). This provides faster but less precise object localisation compared to pixel-wise segmentation.

It is necessary to train these CNNs on a dataset that is comprised of labelled segmented images. For this I will be using Food-Seg103 a dataset consisting of over 7000 images with more than 40 thousand segmentation masks.

The new contribution of this project is the Blend-Seg architecture. Blend-Seg is a lightweight dual-branch architecture that merges object-detection and pixel-level segmentation information. An image is processed in parallel by two branches. One branch uses U-Net to produce refined pixel-level segmentations, and the other uses YOLO to create object-detection information in the form of bounding boxes. The outputs of these branches are then merged and passed through two convolutional layers, where the network learns the optimal way to blend the features produced by both models for a more accurate refined segmentation output.

1.2 Research Questions

This section defines the research questions that serve to guide my work on this final year project

1. **Comparing the effectiveness of different CNN architectures designed for semantic segmentation in food recognition:** This is key to understanding the shortcomings faced by the current architectures. This will involve testing U-Net and DeepLab on the FoodSeg103 dataset to determine their effectiveness.
2. **Evaluating the improvement in segmentation accuracy afforded by the proposed dual-branch Blend-Seg architecture over standard U-Net and DeepLab models:** For this project I have created a custom dual-branch neural network named Blend-Seg. The results from this new technique are then compared to those of U-Net and DeepLab to assess its effectiveness.

1.3 New Contributions

The idea of combining object detection and pixel-level segmentation has existed for a long time. Popular two-stage frameworks like Mask R-CNN work by first localising regions of interest (RoIs) via an object detector and then running a binary segmentation (background

or object) on each RoI (He, Gkioxari, et al. 2017). This process however, is slow and requires substantial computation as each identified RoI needs to be processed individually. Blend-Seg takes a completely different approach to this problem by running a real-time detector (YOLO) and a full-image segmenter (U-Net) in a single pass and then merging their outputs through a lightweight dual-branch fusion module. This allows Blend-Seg to run much faster while still leveraging both precise object localisation and fine-grained mask prediction.

Networks utilising both YOLO and U-Net have also been explored recently. For example, Enhanced TumorNet implements a two-stage pipeline which first uses YOLO to draw bounding boxes around regions containing tumours, and then feeds those bounding box regions through a U-Net model for fine segmentation (Zafar et al. 2024). While providing good results for binary tasks this pipeline mirrors the Mask R-CNN two-stage approach and as such encounters the same problems.

A very recent paper titled "Food image segmentation based on deep and shallow dual-branch network" (Xiao, Y. Li, and Deng 2025) explores using a dual-branch architecture for food segmentation. However, this architecture known as FDSNet is entirely different to the one seen in Blend-Seg. The core concept of Blend-Seg is the fusion of object detection and pixel-level segmentation, whereas FDSNet fuses the output of two pixel-level segmentation models, these being a high powered transformer based model and a more simple standard segmentation CNN.

Blend-Seg differs fundamentally from existing two-stage framework methods by keeping a pretrained YOLO detector and U-Net segmenter entirely independent and then fusing their final outputs through a minimal network. Due to the two models being separate the Blend-Seg architecture is also uniquely modular where either network can be upgraded independently. Unlike two-stage approaches, Blend-Seg's single-pass design scales efficiently to the 103 classes of FoodSeg103 while avoiding the computational overhead associated with region-based processing. To my knowledge, this approach represents the first fusion of two fully independent single-pass network outputs, achieving competitive segmentation performance while preserving inference efficiency.

1.4 Methodology

This section focuses on the project objectives and the steps taken to achieve them:

- **Literature Review:** Conducted a systematic review of relevant papers, covering the history of artificial intelligence, convolutional neural networks (CNNs), and cutting-edge segmentation architectures. This helped identify gaps and refine the research questions.
- **Research Questions Definition:** Observed that existing CNN-based segmentation models performed poorly on food imagery. Focused the investigation on comparing two popular architectures: U-Net and DeepLab.

- **Dataset Preparation:** Selected the FoodSeg103 dataset and integrated it into a Google Colab environment. This provided a realistic and challenging benchmark for training and evaluation.
- **Model Training and Evaluation:** Loaded baseline U-Net and DeepLab models. Conducted a series of experiments varying hyperparameters such as learning rate, number of epochs, loss functions, and regularisation techniques. Validated both models on the same unseen validation set using identical performance metrics.
- **Findings on Over-segmentation:** Analysis revealed that although both models captured object boundaries well, they suffered from over-segmentation, producing numerous small misclassified regions within correct areas (as shown in Figure 2).
- **Development of Blend-Seg:** Based on these findings, designed and implemented the Blend-Seg architecture to directly address over-segmentation issues. Evaluated Blend-Seg under the same experimental conditions to ensure a fair comparison with baseline models.

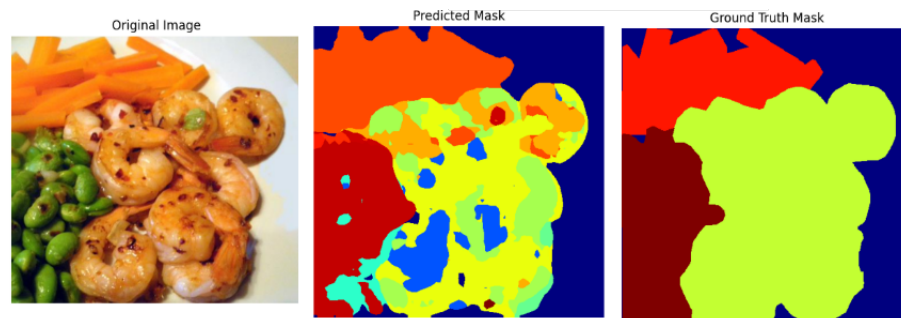


Figure 2: A sample output from one of the DeepLabv3+ experiments. The boundaries of the shrimp, green beans and carrots match closely to the ground truth, but the shrimp region suffers from over-segmentation, with numerous small clusters of misclassified pixels.

1.5 Overview of the Report

- **Chapter 1:** Introduces the project scope, motivation, and research questions guiding this work.
- **Chapter 2:** Presents a comprehensive literature review covering the foundations of AI, convolutional neural networks, and semantic segmentation techniques.
- **Chapter 3:** Details the design and implementation of the Blend-Seg prototype, including the development environment, architectural components, and evaluation setup.
- **Chapter 4:** Describes a series of experiments detailing the training, tuning, and post-processing of DeepLab, U-Net, YOLO, and Blend-Seg models and analyses their performance on the FoodSeg103 dataset.
- **Chapter 5:** Discusses the key findings, assesses the contributions of the work, and outlines directions for future research.

1.6 Motivation

Dietary-related illnesses have become increasingly prevalent in recent decades, and consistent tracking of food intake over a 24-hour period has been shown to help users combat underlying health problems (Lo et al. 2021). While smartphone apps that analyse meal photos to estimate nutritional content offer a promising way to reduce the burden of manual calculation, their accuracy is severely limited by poor semantic segmentation of food items. This research therefore focuses on improving segmentation performance as a critical foundation for any reliable, automatic diet-tracking solution.

2 Literature Review

This section covers a wide range of topics from the basics of artificial intelligence and machine learning to more in-depth looks at some of the cutting-edge techniques and networks that exist today. All this knowledge is necessary context for understanding the make-up of semantic segmentation CNNs and how to go about evaluating their performance.

2.1 Overview of Artificial Intelligence

Artificial intelligence (AI) is defined as "the science and engineering of making intelligent machines" (Manning 2022). AI has a wide variety of uses including image recognition, smart speakers and self-driving cars (Haenlein and Kaplan 2019).

The term Artificial Intelligence was first officially coined in 1956 when Marvin Minsky and John McCarthy hosted the Dartmouth Summer Research Project on Artificial Intelligence (DSRPAI). The goal of this was to develop a new research area focused on creating machines that would be able to simulate human intelligence. (Haenlein and Kaplan 2019) Since then the field of AI has grown rapidly enabling the development of the cutting-edge technologies we see today.

2.2 Artificial Neural Networks

Artificial neural networks (ANNs) are mathematical models inspired by the functioning processes of the human brain (Grossi and Buscema 2007). ANNs are constructed of layers of interconnected processing elements known as neurons, which are organised into 3 main types, input layers, hidden layers and output layers. Data is passed into the input layer where each neuron receives a weighted signal that mimics the strength of synaptic connections in the brain. The hidden layers perform non-linear computations on the input which produce the neuron's output. Typically, the output of one layer then serves as the input for the next layer. Learning in the ANNs is achieved by adjusting the weights of these connections during training, strengthening the pathways that lead to accurate outputs. (Steven Walczak and Cerpa 2006)

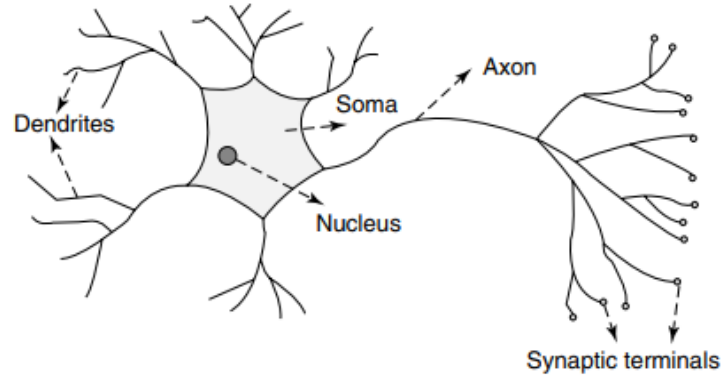


Figure 3: Mammalian neuron (Abraham 2005)

2.2.1 Perceptron

The perceptron was created by Frank Rosenblatt a psychologist from Cornell University in the 1950s and was designed to recognise the letters in the alphabet. He based the design on the human nervous system. It used both analog signals which change continuously and discrete signals which take fixed values such as 0 and 1. It then used a threshold that turned analog signals into discrete outputs.(Fradkov 2020) A basic diagram of a single-layer perceptron can be seen in Figure 4. Perceptrons have binary outputs, so a single perceptron can only be trained to recognise a single letter (Minsky and Papert 1969). For example, if a perceptron was trained to recognise the letter "A" it could only return a true or false value representing if the inputted letter was an "A". Despite its simplicity the perceptron introduced key concepts like weighted inputs and thresholds, which would become the precursor to modern activation functions. However, its limitations in handling non-linear problems such as an inability to solve the exclusive-OR problem drew criticism (Macukow 2016), highlighting the need for further innovation in neural network design.

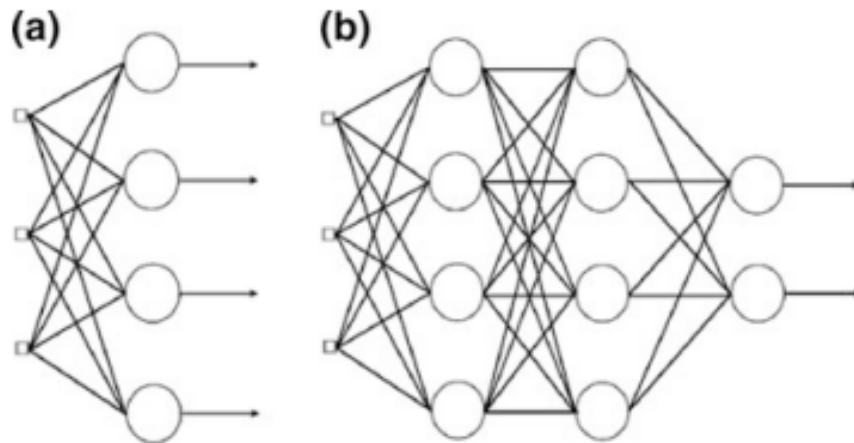


Figure 4: (a) diagram of a single-layer perceptron with 4 outputs (b) diagram of a multilayer perceptron with 2 outputs (Taud and Mas 2018)

2.2.2 Multilayer Perceptron

The first improvement came in the form of the Multilayer Perceptron (MLP). MLPs enabled the handling of non-linear problems by introducing more layers to the standard perceptron architecture. The first layer is the input layer, the last layer is the output layer and all the other layers are known as hidden layers. The key design feature is that the output for one layer becomes the input for the next layer (Taud and Mas 2018). This architecture enabled the modelling of much more complex relationships. However, early attempts to train these MLPs were limited as there was no efficient method to update the weights across multiple layers. It was not until the introduction of the backpropagation learning algorithm that the potential of these MLPs was truly realised sparking "a revitalisation of research and development efforts in artificial neural networks" (S. Walczak 2018)

2.2.3 Backpropagation

The back propagation learning algorithm was developed in 1974 by Paul Werbos, although it was not fully appreciated until 1986 with the publication of the book *Learning Internal Representation by Error Propagation* by Rumelhart, Hinton and Williams (Macukow 2016). Backpropagation is designed to optimise the weights by minimising the error between the predicted output and the target. This works by updating the weights using a method called gradient descent that reduces the error over time (Rumelhart et al. 2013.)

The backpropagation algorithm is split into two parts: the forward pass and the backwards pass. During the forward pass, the input data passes through the network layer by layer having its value adjusted by weights and activation functions until it reaches the output layer. This outputted layer is then compared to the target and the error is calculated based on the difference between these values. In the backward pass, the algorithm calculates the gradient of the error for each weight. The weights are then updated based on this gradient proportional to a pre-defined learning rate (Rumelhart et al. 2013.)

The backpropagation formula can be seen here:

$$w_{ij} \leftarrow w_{ij} - \eta \frac{\partial E}{\partial w_{ij}}$$

Where:

- w_{ij} is the weight connecting neuron i to neuron j ,
- η (eta) is the learning rate,
- E is the loss (error) function, and
- $\frac{\partial E}{\partial w_{ij}}$ is the gradient of the loss with respect to that weight.

As the network repeats this process of forward and backward passes the weights are slowly refined reducing the difference between the output and target values. This enables the network to learn complex and non-linear patterns in the data. This was a major breakthrough in

the field of ANNs and made the development of modern advancements such as convolutional neural networks possible.

2.3 Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a deep learning method that takes an input image and assigns importance to the distinctive features of that image, such as edges or textures in the form of learnable weights and biases in order to differentiate them from one another (Bhatt et al. 2021).

Like any standard neural network model, CNNs are based on neurons that are organised in layers allowing them to learn hierarchical representations. The initial layer is the input layer containing the raw data, and the final layer is the output layer that returns whatever the target for the model is. In between there are a number of hidden layers that transform the feature space of the input so that it matches the desired output (Kattenborn et al. 2021).

CNNs consist of an input, an output, and 3 different types of layers: these being the convolutional layers, pooling layers, and fully connected layers. Additionally, activation functions are applied after each convolution to ensure non-linearity and dropout layers are included to prevent overfitting (O'Shea and Nash 2015). Overfitting is when a model begins to fit too well to the training data, leading to high accuracy scores when training but low accuracy scores when validating on unseen data (Yamashita et al. 2018).

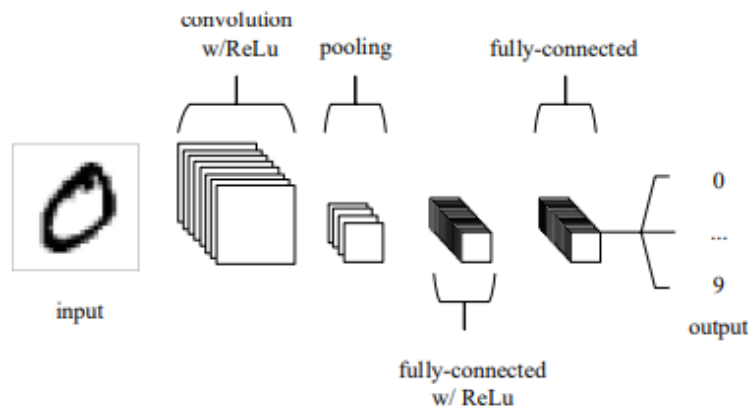


Figure 5: Diagram of a simple CNN architecture (O'Shea and Nash 2015)

2.3.1 Convolutional Layer

The convolution layer is responsible for performing feature extraction. The layer consists of a linear mathematical computation called a convolution operation and a non-linear activation function (Yamashita et al. 2018)

In the convolution operation, kernels which are small arrays of numbers are spread along the entire depth of the input and are used to detect different patterns such as edges or

textures. Each kernel scans a small section of the input image performing a dot product between the kernel and that section. The result of this is then added to a feature map (J. Guo et al. 2018) Figure 6 shows this process visually. When a feature is detected the neurons "fire", meaning they produce a high value which indicates that the feature is present at that location. A different kernel is used for each feature map, all of which are stacked along the depth dimension to form the full output volume from the convolutional layer (O'Shea and Nash 2015). Two important hyperparameters for the convolution operation are the size of the kernels and the number of kernels. A typical size for a kernel is 3x3 which means the kernel has a height and width of 3 pixels, but they can be a number of sizes such as 1x1, 5x5 or 7x7. The bigger kernels can capture more abstract global features in the image at a higher computational cost. The number of kernels is arbitrary and determines the depth of the output feature maps (Yamashita et al. 2018).

To reduce a models complexity it's important to optimise the output volume. This is done with the 3 hyperparameters depth, stride and zero-padding (Aloysius and Geetha 2017)

Depth is the number of kernels used at each layer. **Stride** refers to how many pixels a kernel shifts as it moves across the image for example a stride of 1 means it moves at 1 pixel every time leading to a lot of overlapping kernels, and a stride of 2 means it moves 2 pixels every time which reduces the size of the feature map. **Zero-padding** adds extra zeros around the border of the input allowing control of the output size (O'Shea and Nash 2015)

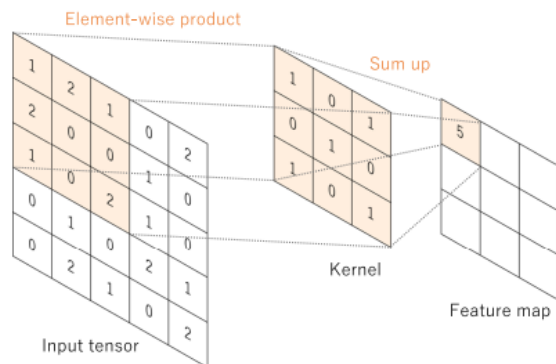


Figure 6: Visual representation of a convolutional layer (Yamashita et al. 2018)

2.3.2 Activation Function

The outputs of the linear convolution operation are then passed through a nonlinear activation function. Their purpose is to compute the weighted sum of input and biases to determine whether a neuron should fire or not (Nwankpa et al. 2018). In the past smooth functions such as the hyperbolic tangent or sigmoid functions were used as they most closely resembled biological neuron behaviour, but nowadays the most common activation function is the Rectified Linear Unit (ReLU) which works by outputting zero for all negative values and computing the function: $f(x) = \max(0, x)$ (Yamashita et al. 2018). The difference between these functions can be clearly seen in Figure 7. ReLU is preferred because it avoids

the vanishing gradient problem which is when the gradients become so small during back-propagation that the weight updates become negligible causing learning to dramatically slow down or stop altogether.

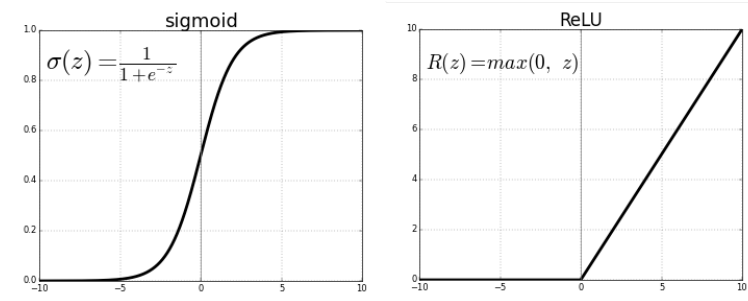


Figure 7: Graph of sigmoid and ReLU activation functions

2.3.3 Pooling Layer

The purpose of a pooling layer is to slowly decrease the dimensionality of the representation therefore also reducing the number of parameters and computational complexity (O'Shea and Nash 2015). The most common type of pooling is max pooling, this works by taking a patch of the feature map outputting the maximum value of that patch and discarding all other values (Yamashita et al. 2018). Figure 8 below provides an example of a max pool layer downsampling a feature map by a factor of 2

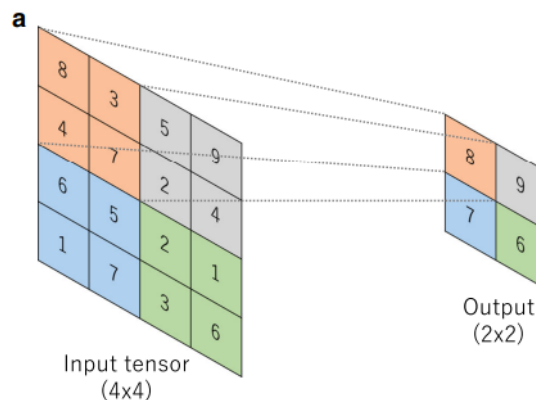


Figure 8: Visual representation of a 2x2 max pooling operation with a stride of 2 (Yamashita et al. 2018)

Another type of pooling of note is global average pooling which uses a more extreme form of downsampling. All of the feature maps are downsampled into a 1x1 array made up of the average of all of the elements in the feature map. This is typically done only once before the fully connected layers. This allows for the reduction of learnable parameters while also allowing the CNN to take in inputs of variable sizes (Yamashita et al. 2018).

2.3.4 Fully Connected Layer

In a fully connected layer every input is connected to every output by a learnable weight. The output of the pooling layers is flattened meaning it is transformed into a one-dimensional array of numbers before being input into one or more fully connected layers, which are then mapped to the final outputs of the network. Each fully connected layer is followed by a non-linear activation function (Yamashita et al. 2018).

2.3.5 ResNet

Back in 2016, all the research showed that the depth of a CNN was of crucial importance when it came to the quality of results. At the time all of the top performing networks utilised "very deep" models with depths between 16 and 30 (Simonyan and Zisserman 2014). Due to these trends the temptation could be to just stack as many layers as possible in hopes of increasing performance. Historically, this was not possible due to the infamous vanishing gradients problem as well as degradation in training and test accuracy as the depth increased. This degradation occurred due to the optimisation methods of the time, struggling to train very deep networks effectively. (He, X. Zhang, et al. 2016)

In 2016 there was a breakthrough on this problem when Kaiming He and his team published the paper "Deep Residual Learning for Image Recognition" and introduced ResNet to the world. ResNet which is short for residual network, works by utilising something known as residual learning. How this works is instead of fitting every layer to a desired underlying mapping the layers are fitted to a residual mapping which is the difference between the input and the desired mapping. Fitting to the residual mapping is much simpler allowing for the construction of far deeper networks. (He, X. Zhang, et al. 2016). This is made possible through shortcut connections which skip one or more layers allowing information to flow freely through the network without adding any additional computational complexity. (He, X. Zhang, et al. 2016).

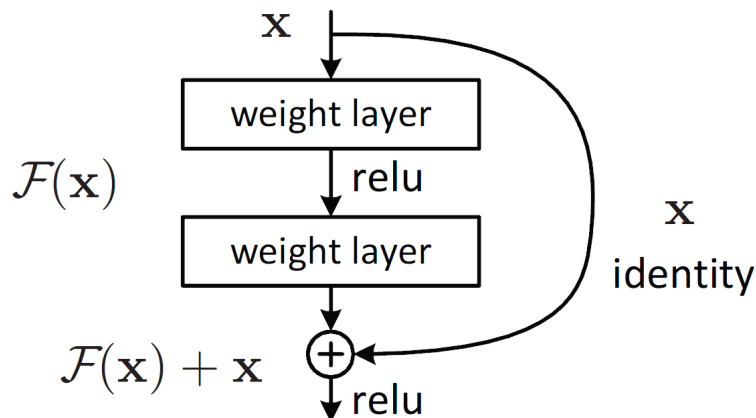


Figure 9: A residual block showing the input X utilising a shortcut connection to bypass the weight layers (He, X. Zhang, et al. 2016)

The advantages of this approach were shown in 2015 when a ResNet model won 1st place in the ILSVRC classification competition. This model consisted of 152 layers and was the deepest network ever used on the benchmark testing dataset ImageNet (He, X. Zhang, et al. 2016). This success validated the use of residual learning and was a huge step forward in the fields of computer vision and deep learning. The reason I have chosen to focus on ResNet in this section is that many segmentation models such as U-Net have incorporated its residual learning techniques to enhance feature extraction and training efficiency (Siddique et al. 2021).

2.3.6 You Only Look Once (YOLO)

You Only Look Once (YOLO) is a real-time object detection CNN introduced by Joseph Redmon and colleagues in 2016 (Redmon et al. 2016). YOLO tries to return a label for each of the objects in the input image. To do this it is trained on a large training set of images annotated with labelled bounding boxes. Through this training, it learns how to correctly apply bounding boxes around each of the objects in a given image and then assign labels to those boxes (Redmon et al. 2016). YOLO also looks at the whole image in a single pass through the network, this is much simpler than most other networks. For example, in a benchmark of identical hardware YOLO processed 45 images a second while R-CNN, another object detection CNN took 40 seconds to process just one image (Redmon et al. 2016). YOLO is designed to function in real time and prioritises returning labels fast and efficiently (Redmon et al. 2016) at the cost of the detailed boundary information that pixel-wise classification provides.

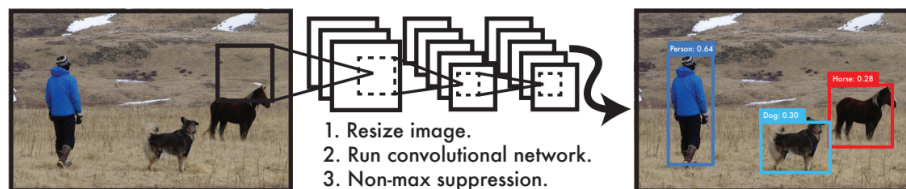


Figure 10: An example of how YOLO applies bounding boxes (Redmon et al. 2016)

YOLO works by first dividing the image into an even grid. If the centre of an object falls into a grid cell, that grid cell is responsible for detecting that object (Redmon et al. 2016). Each grid cell predicts a confidence score which reflects the likelihood of an object being present in that cell. It then assigns class probabilities to each detected object representing what the model believes the object is. These numbers are then used to determine where the bounding boxes are drawn in the output and what labels are assigned to them as seen in Figure 11. (Redmon et al. 2016)

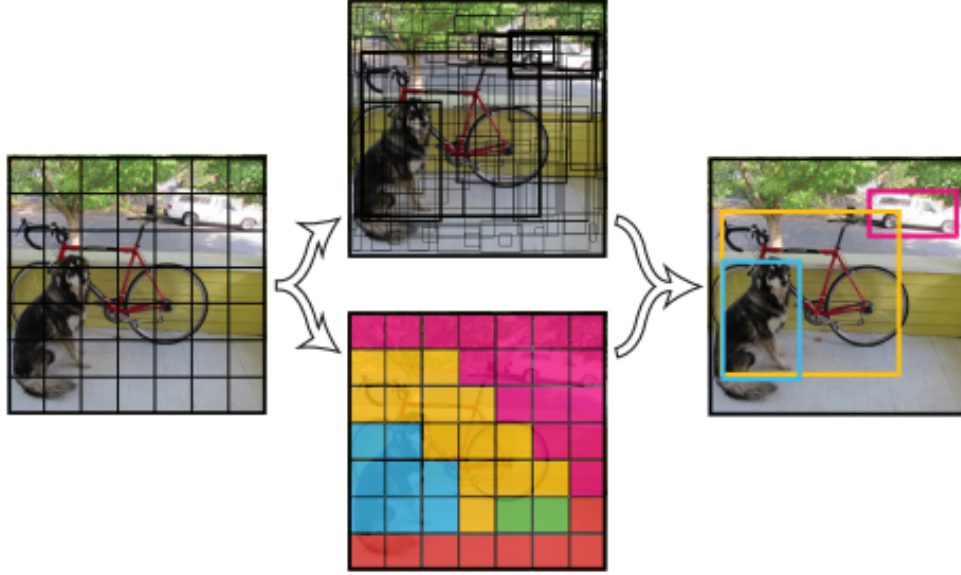


Figure 11: Representation of how YOLO divides the image into an even grid, predicts the object in each grid cell and uses that information to determine the bounding boxes in the output. (Redmon et al. 2016)

YOLO is designed to work in real time and is most suitable for tasks such as video analysis and autonomous systems but its key limitations are a lack of accuracy and difficulty differentiating small objects that appear close together (Redmon et al. 2016). The Blend-Seg architecture proposed in this paper is designed to take advantage of YOLOs strengths while covering its weaknesses with the use of a secondary semantic segmentation CNN.

2.4 Semantic Segmentation

Semantic segmentation is the process of segmenting an image into different parts and objects where every pixel of the image is assigned a specific label. It is utilised in many real-world applications such as video surveillance, autonomous driving and medical imagery (Mo et al. 2022). When given an image, a segmentation algorithm should output the groupings of the pixels that belong together. We can see this in Figure 12 below, all the pixels belonging to a motorcycle are coloured green to show they belong together, whereas the riders and the audience are coloured pink as they are correctly being identified as people (J. Guo et al. 2018)

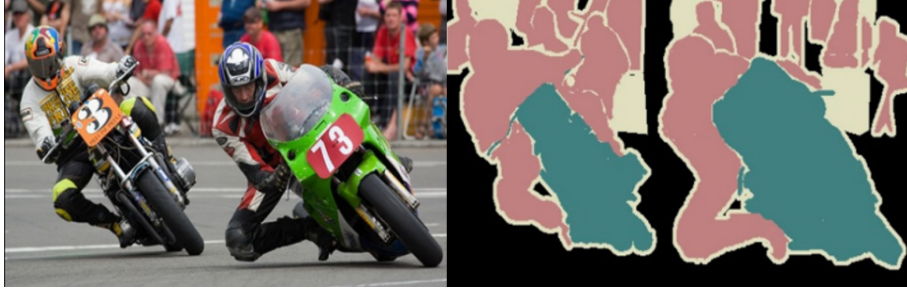


Figure 12: Segmentation for motorcycle racing image (J. Guo et al. 2018)

Historically, researchers used random forest and conditional random field for semantic segmentation, these are probabilistic models that look at neighbouring pixels to determine what label to apply (Mo et al. 2022). The first CNN for semantic segmentation was introduced in 2015 by Jonathan Long, Evan Shelhamer and Trevor Darrell in their paper titled "Fully Convolutional Networks for Semantic Segmentation" and was named the Fully Convolutional Network (FCN) (Long, Shelhamer, and Darrell 2015). The FCN showed a 20% improvement on the Pascal Visual Object Classes 2012 dataset compared to previous segmentation techniques (Mo et al. 2022) highlighting the bright future of CNNs in semantic segmentation.

2.4.1 U-Net

U-Net is a CNN architecture that was originally created to analyse medical images but is now used in many fields. It gets its name from its unique U-shaped architecture consisting of a contracting path on the left and a symmetric expansive path on the right (see Figure 13) (Ronneberger, Fischer, and Brox 2015).

Each step in the contracting path applies two successive 3×3 convolutions, each followed by a ReLU activation, and then a 2×2 max-pooling operation with stride 2. With every down-sampling step, the resolution of the feature maps is halved while the number of feature channels doubles, allowing the network to capture increasingly abstract features at lower resolutions (Ronneberger, Fischer, and Brox 2015).

In the expansive path, each stage begins with a 2×2 transpose convolution (sometimes called up-convolution) that doubles the spatial dimensions and halves the number of channels. Then this up-sampled map is concatenated with the corresponding feature map from the contracting path via the skip-connection shown by the grey arrows in Figure 13. Finally, two more 3×3 convolutions (each with ReLU) refine the fused features. This concatenation of feature maps restores high-resolution details lost during pooling by directly reintroducing encoder activations into the decoder (Ronneberger, Fischer, and Brox 2015).

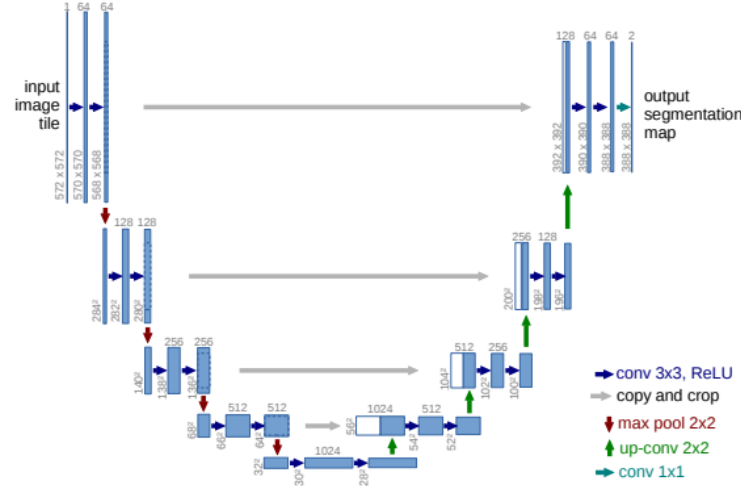


Figure 13: U-Net architecture with contracting path (left), expansive path (right), and skip connections that copy and crop encoder feature maps into the decoder (Ronneberger, Fischer, and Brox 2015).

One of the things that makes U-Net special is its ability to create highly detailed segmentation maps from limited training samples. This makes it particularly useful in the medical field, where properly labelled images are often scarce. It is also much faster to train than most other segmentation models (Siddique et al. 2021). As such, U-Net has become a very popular architecture.

2.4.2 DeepLab

DeepLab is an advanced deep learning model built for semantic segmentation, introduced in 2015 by Liang-Chieh Chen and other researchers from Google. While developing DeepLab these researchers set to solve 3 key issues that occurred when attempting to use Deep Convolutional Neural Networks (DCNNs) for semantic segmentation these being (1) reduced feature resolution, (2) existence of objects at multiple scales, and (3) reduced localisation accuracy due to DCNN invariance. (Chen et al. 2017).

In image classification the global information of the image is more important than retaining the precise spatial details. Due to this traditional DCNNs use downsampling which results in low-resolution feature maps. This leads to issues when attempting semantic segmentation, as exact spatial details are important, meaning higher resolution feature maps are needed. To solve this problem, DeepLab removes downsampling from the last few pooling layers and instead upsamples using atrous convolutions (Now known more commonly as dilated convolutions). (Chen et al. 2017) This works by introducing holes between the non-zero kernels as seen in Figure 14. This increases the kernel’s field of view without requiring any extra computational power. This allows DeepLab to capture more context while maintaining high-resolution feature maps which is necessary for accurate segmentation. (Chen et al. 2017)

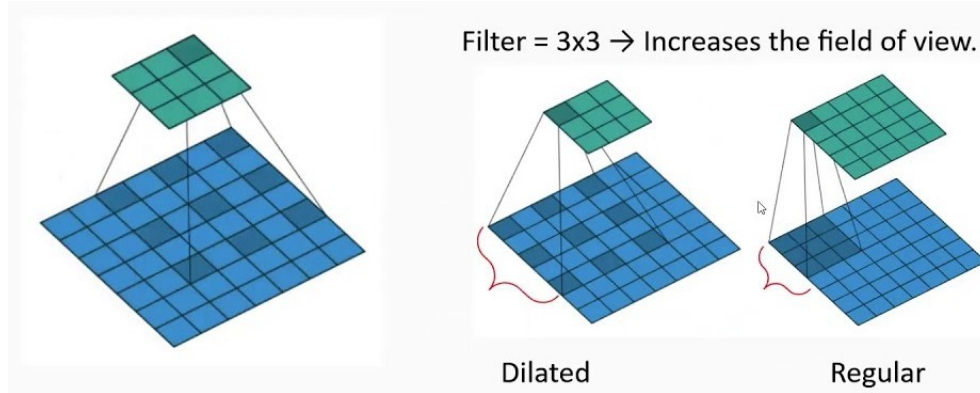


Figure 14: Visual representation of an atrous convolution (K. Zhang et al. 2019)

The next problem DeepLab sets out to solve is how to deal with objects at different scales such as a close-up of a person and a person in the background. The standard way this is dealt with is by presenting the DCNN with rescaled versions of the same image and getting an average for the feature map. This however, is computationally expensive, so instead of rescaling the image DeepLab resamples the feature maps by using multiple parallel atrous convolutional layers with different sampling rates. This is a much more computationally efficient way of capturing image context at multiple scales, they named this technique atrous spatial pyramid pooling (Chen et al. 2017).

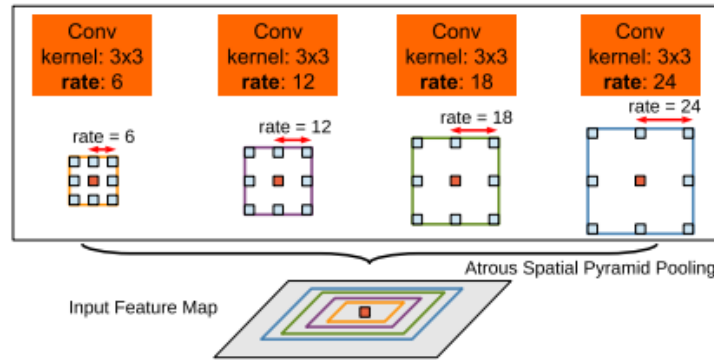


Figure 15: Visual representation of Atrous Spatial Pyramid Pooling the effective Field-Of-Views are shown in different colours (Chen et al. 2017)

The third issue DeepLab aims to address is that DCNNs are designed to be unaffected by spatial transformations such as the object's position or orientation. This limits their ability to capture fine spatial details and boundaries which are crucial in pixel-level tasks such as semantic segmentation. To mitigate this problem DeepLab incorporates Fully Connected Conditional Random Fields (CRFs). CRFs are probabilistic models that predict labels by considering the dependencies between neighbouring outputs, such as adjacent pixels. An example of DeepLabs results before CRF and after CRF can be seen in Figure 16.

This implementation improves the model’s ability to capture fine details by combining high-level class predictions from the DCNN with low-level image information like pixel and edge interactions (Chen et al. 2017). This improves the accuracy of segmentation while also being computationally efficient.



Figure 16: Results of DeepLab on an Image Before CRF and After CRF (Chen et al. 2017)

2.4.3 Mask R-CNN

Mask R-CNN is a popular convolutional neural network architecture specifically designed for instance segmentation. While semantic segmentation models such as U-Net and DeepLab assign each pixel to a class without differentiating individual objects. Instance segmentation models identify each distinct object separately and generate individual segmentation masks for them (He, Gkioxari, et al. 2017).

Introduced by Kaiming He and colleagues in 2017, the architecture of Mask R-CNN consists of two main stages. In the first stage, the network scans the image and generates candidate regions that might contain objects. In the second stage, each of these candidate regions is processed independently by three parallel branches. The first branch classifies the object type, the second branch predicts the bounding box coordinates to localise the object and the third branch produces a binary mask that segments the precise shape of each object within its bounding box (He, Gkioxari, et al. 2017).

Mask R-CNN has proven highly effective in various image analysis and segmentation tasks. However, since all candidate regions need to be processed separately through multiple parallel branches, Mask R-CNN is much slower and more resource-intensive than purely semantic segmentation models (He, Gkioxari, et al. 2017) like U-Net and DeepLab.

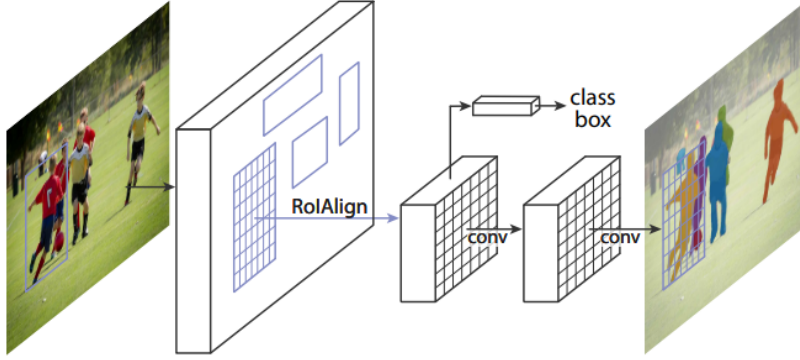


Figure 17: The Mask R-CNN framework for instance segmentation (He, Gkioxari, et al. 2017)

2.4.4 Two Stage Detection + Segmentation Techniques

Object-detection CNNs excel at quickly localising distinct items, whereas semantic-segmentation CNNs shine at pixel-level labelling yet often mislocalise whole objects. The relationship between these two techniques is interesting as their specific strengths and weaknesses appear to complement each other. The key idea this report aims to explore is the possibility of creating an architecture that can take advantage of this potential synergy by fusing these two methods in a way that allows them to cover each other’s weaknesses.

In the field of medical imagery analysis, several recent studies have been done on architectures that utilise both object detection and semantic segmentation techniques. For example the paper ”Enhanced TumorNet: Leveraging YOLOv8s and U-net for superior brain tumor detection and segmentation utilizing MRI scans” (Zafar et al. 2024) proposes a two-stage pipeline where a YOLO model first localizes tumour regions with a bounding box, then a U-Net model performs binary tumour vs background segmentation (Zafar et al. 2024). Enhanced TumorNet achieves state-of-the-art results in both accuracy (98.6%) and recall (98%) outperforming existing models by between 1-8% (Zafar et al. 2024).

While this specific method works well in binary medical tasks, it would not translate well to food image analysis, which can have over 100 distinct classes. For example, FoodSeg103 has 103 unique food labels; to apply the same technique as TumorNet you would need to train 103 distinct U-Net models capable of binary segmentation. Additionally, food images are more cluttered with food items often overlapping, which this technique would be unable to handle. However, the high results demonstrated by TumorNet are encouraging for the basic principle of an architecture that leverages both YOLO and U-Net. It would just need to be an architecture designed with the unique challenges of food segmentation in mind.

For example, the key difference in the Blend-Seg architecture proposed by this report is that instead of using a pipeline where the input passes through YOLO and then U-Net, Blend-Seg runs both YOLO and U-Net in parallel and then concatenates the outputs. This technique

is much more natural for the large number of classes and overlapping items inherent to food segmentation.

2.4.5 Performance Metrics

Since a key part of this project focuses on comparing the performance of various semantic segmentation CNNs it becomes important to define how we will be evaluating these architectures' performances.

One of the most commonly used segmentation-based metrics is the **Dice Similarity Coefficient (DSC)**. This gives a value for the overlap between the predicted segmentation and the ground truth (the pre-segmented training data). The formula for DSC is as follows:

$$\text{DSC} = \frac{2 \times TP}{2 \times TP + FP + FN}$$

Where:

- **TP (True Positives):** is the number of pixels correctly identified as part of the target object
- **FP (False Positives):** is the number of pixels incorrectly identified as part of the target object
- **FN (False Negatives):** is the number of pixels that were part of the target object but were incorrectly labelled

A DSC score close to 1 indicates a high degree of overlap between the prediction and the ground truth, whereas a score close to 0 indicates poor overlap. This metric is popular as it provides a figure that accounts for both false positives and false negatives. This is particularly useful as the images being segmented are usually heavily imbalanced with far more background pixels than target pixels which can heavily bias other performance metrics. (Müller, Soto-Rey, and Kramer 2022) For example, this traditional accuracy formula below

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FN + FP}$$

Is not a recommended performance metric to use as simply labelling the entire image as background can often result in accuracy scores of over 90% due to the data imbalance (Müller, Soto-Rey, and Kramer 2022)

Another commonly used segmentation-based metric is the **Intersection over Union (IoU)**, also known as the Jaccard Index (Taha and Hanbury 2015). This metric measures the ratio

of the overlap between the predicted segmentation and the ground truth to their union. The formula for IoU is as follows:

$$\text{IoU} = \frac{TP}{TP + FP + FN}$$

Where:

- **TP (True Positives):** the number of pixels correctly identified as part of the target object
- **FP (False Positives):** the number of pixels incorrectly identified as part of the target object
- **FN (False Negatives):** the number of pixels that were part of the target object but were incorrectly labelled

An IoU score close to 1 indicates a high degree of overlap between the prediction and the ground truth, whereas a score close to 0 indicates poor overlap. Compared to DSC, IoU is slightly harsher penalising boundary mismatches. As a result, it is often used as the benchmark metric for semantic segmentation tasks (Taha and Hanbury 2015).

2.5 Training Data

In order to train these semantic segmentation CNNs it is necessary to use a training dataset that consists of a large number of accurately segmented images. Creating these datasets however, is a laborious and painstaking job meaning their availability is limited. As a result, researchers are often required to construct their own.

2.5.1 Creating a Training Dataset

The first step in creating a segmented dataset is sourcing a collection of high-quality images. This is typically done by using an existing public non-segmented dataset. Segmenting the images is then usually accomplished with the combination of a pretrained semantic segmentation CNN that gives a rough initial output and a human who is responsible for refining the segmentation mask and verifying the assigned labels (Kontogianni et al. 2020). One of the big obstacles in this is due to the limited availability of segmented datasets, the pretrained CNNs often lack the relevant training data required to produce accurate initial outputs (Xu et al. 2016). Due to this, it becomes necessary to use segmentation techniques designed to help with unseen data

One example of this is talked about in the paper "Deep Interactive Object Selection" (Xu et al. 2016). The paper suggests an approach where the user can make a number of positive and negative clicks on the image to guide the CNNs segmentation. Positive clicks are used

to identify the object the user is trying to segment (the foreground) while negative clicks identify the background. This allows the user to easily and quickly refine the segmentation mask (Xu et al. 2016). Figure 18 shows a large improvement in precision after only three user inputs.



Figure 18: output of the CNN as number of user clicks increases (Green dots representing positive clicks, red dots representing negative) (Xu et al. 2016)

The segmentation outputs are then refined using graph cut. Graph cut is an optimisation method where an image is represented as a graph, and the goal is to partition the image into foreground and background regions by minimising a cost function (Boykov and Jolly 2001). The graph cut framework uses positive and negative clicks from users as constraints to help guide the algorithm in accurately adjusting the segmentation boundaries. The use of graph cut makes a big difference in the accuracy of the boundaries as we can see in the comparison between Figure 18 and Figure 19



Figure 19: output of the CNN after graph cut refinement (Xu et al. 2016)

However, when constructing a training data set highly accurate segmentations are a necessity and unfortunately these algorithms and techniques are not perfect. As a result it is still necessary for a human to check and refine each and every segmentation mask before they can be added to a training data set. These methods do however speed up the process dramatically

2.5.2 Food-Seg103

For this project, it will be necessary to have a segmented food image dataset to train my models. Luckily, one already exists in the form of Food-Seg103. This data set consists of 7,118 images and more than 40K masks covering 103 food ingredients. The source images for Food-Seg103 come from the existing dataset Recipe1M. This data set consists of one million cooking recipes along with 13 million associated images. This data set was particularly useful as a base for a segmented data set because all recipes contained a list of ingredients. This ingredient list was used as auxiliary information for the initial semantic segmentation models and helped improve the accuracy of their segmented outputs. (Wu et al. 2021)

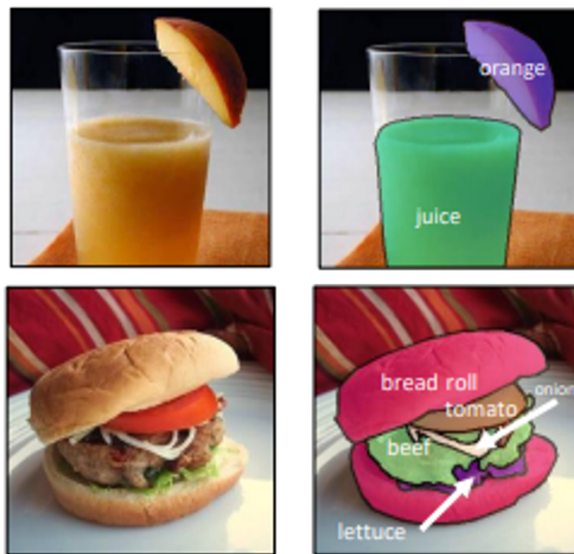


Figure 20: Examples of both simple and complex segmented images in the Food-Seg103 database (Wu et al. 2021)

To apply the segmentation masks, the researchers engaged an outside company to annotate and apply a pixel-wise mask to each ingredient. The annotators were asked to ignore regions with area covering less than 5% of the total image. Once all the masks were received the researchers performed a final refinement of the data. This included correcting any mislabelled data, removing any category with less than five labels and combining categories such as orange and citrus (Wu et al. 2021). Once these steps were completed, the dataset was refined and ready to be used for training.

Recent work has evaluated how different CNN architectures perform on the Food-Seg103 dataset. Traditional convolutional models such as FPN (ResNet-50), PVT-S, DeepLabV3+, and CCNet (ResNet-50) achieve mean Intersection over Union (mIoU) scores of 27.8%, 31.3%, 34.2%, and 35.5%, respectively (Wu et al. 2021). More modern transformer-based models like SeTR-MLA and FoodSAM have pushed performance further to 45.1% and 46.4% (Papers With Code 2025). However, these gains typically come at the cost of much higher

computational requirements. Overall, the relatively modest scores across a range of architectures reflect the difficulty of food segmentation tasks, where overlapping ingredients, diverse textures, and visual ambiguity remain persistent challenges.

3 Prototype

This section provides an overview of my custom Blend-Seg segmentation model. Covering the development environment, architectural details, implementation and the evaluation of the results.

3.1 Introduction

Accurate segmentation of food items in images is crucial for practical dietetics applications, such as portion size measurement and caloric estimation (A. Rouhafzay, G. Rouhafzay, and Jbilou 2025). Traditional segmentation architectures like the ones discussed in the literature review (U-Net and DeepLab) can provide high-quality pixel-level segmentation, but they can struggle with the complex boundaries and overlapping objects present in food images (Wu et al. 2021). On the other hand, object detection methods such as YOLO excel at accurately locating and labelling objects using bounding boxes but lack pixel-level segmentation masks.

The complementary strengths and weaknesses of these two types of models inspired the creation of Blend-Seg, a custom dual-branch segmentation model which fuses the output of pretrained U-Net and YOLO models.

3.2 Development Environment

All development and experimentation for Blend-Seg was conducted on Google Colab using a T4 GPU with 15.0 GB of GPU RAM and 51.0 GB of System RAM. The Colab environment ran Python 3.11.12 and PyTorch 2.6. Additional libraries included `segmentation_models_pytorch` for the U-Net-based segmentation pipeline. Ultralytics for the YOLO bounding-box detection. NumPy, Pandas and scikit-image for data manipulation and Matplotlib for visualisation. A fixed random seed of 42 was set at the start of each training session to enhance reproducibility. All code was saved in a private GitHub repository to facilitate version control and safe rollbacks.

3.3 Architecture Details

The Blend-Seg model fuses outputs from two separate neural networks, U-Net and YOLO, to produce a refined food segmentation result. A diagram of the network’s architecture can be seen in Figure 21. To start, an input image is passed into both the U-Net and YOLO branches in parallel:

- **U-Net** scans the input image and for each pixel produces a set of 104 logits one for each class in the dataset. Logits are raw un-normalised values representing how likely the model thinks that pixel belongs to a given class. These values are stored as a stack of 104 two-dimensional arrays (each representing one of the 104 classes), where each array contains the confidence score for the given class at every pixel. This stack of arrays is known as the U-Net logits.

- **YOLO** scans the input image and generates bounding box predictions for each detected object. Each bounding box contains a confidence score between 0-1 for each class in the dataset. In order to mimic the structure of the U-Net logits, these confidence scores are used to create 104 two-dimensional arrays, one for each class in the dataset. In these arrays, any pixel inside a bounding box is assigned that bounding box’s confidence score for the given class; all pixels outside the bounding box are given a value of 0. Just like U-Net, these 104 arrays are then stacked on top of each other. This stack of arrays is known as the YOLO map.

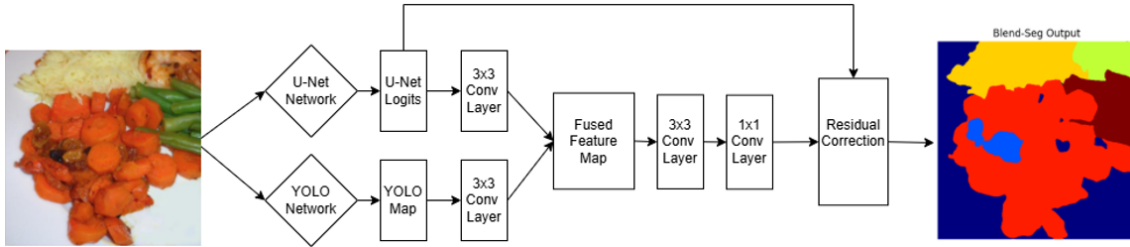


Figure 21: Blend-Seg Architecture

These stacks are then both passed through a 3×3 convolution layer with batch normalisation and a ReLU activation layer to reduce dimensionality ($104 \rightarrow 64$) and align each stack’s values into more comparable ranges while also providing some early feature extraction. After this, the two stacks are concatenated into a single fused feature map with 128 channels.

This fused feature map now contains both YOLO’s accurate object localisation knowledge and U-Net’s pixel-level segmentation information. Rather than predicting the final segmentation from scratch, Blend-Seg learns the difference between the original U-Net logits and the ground truth segmentation masks. This difference is known as a residual correction. This makes training easier and more stable, as the fusion module can learn to ignore regions where U-Net already excels.

To generate that residual correction, the fused feature map is first passed through an additional 3×3 convolutional layer with batch normalisation and a ReLU activation layer. This layer mixes information from both branches, allowing the network to learn where it’s best to trust U-Net’s pixel-level cues or YOLO’s object-level cues. The final 1×1 convolutional layer produces the residual correction by reducing the current 128 channels down to 104 channels, matching the dimensionality of the original 104-channel U-Net logits. The residual correction and the U-Net logits are then added together to produce the final segmentation output by Blend-Seg.

This architecture allows Blend-Seg to leverage YOLO’s strong object localisation abilities while maintaining U-Net’s accurate segmentation results. Additionally, since both models perform classification, Blend-Seg is not bottlenecked by YOLO’s predictions, as the U-Net branch can correct incorrect YOLO labels.

3.4 Implementation

This section provides a walk-through of the key implementation present in the Blend-Seg prototype providing explanations of both what the code does and why it was structured this way. (full implementation can be seen here at [GitHub](#))

3.4.1 Class Mappings and Data Loading

The class mapping and data loading implementation can be seen in Figure 22. First, the class_mappings CSV is loaded. This contains two columns, "Class ID" and "Class Name". This allows the ability match up the IDs used by models with the human-readable food names, which is important for visualisation and analysis. For the U-Net model the ID 0 represents the background class and the first food label starts at 1. However, YOLO does not contain a background class, meaning the first food label starts at 0. This creates a mismatch between the YOLO and U-Net IDs which is corrected by creating the yolo_to_unet dictionary which is used to correctly match up the IDs.

Next, the 3 training parquet files are loaded and concatenated into a single DataFrame full_df

```
# Mappings & Data Loading
csv_path = "/content/drive/MyDrive/foodseg103/class_mappings.csv"
df_mapping = pd.read_csv(csv_path)

yolo_to_unet = {}
id_to_food = {0: "background"}
for _, row in df_mapping.iterrows():
    class_id = int(row["Class Id"])
    if class_id > 0:
        yolo_id = class_id - 1 # YOLO food labels start at 0
        unet_id = class_id     # U-Net food labels start at 1
        yolo_to_unet[yolo_id] = unet_id
        id_to_food[unet_id] = row["Class Name"]

print("YOLO to U-Net mapping:", yolo_to_unet)
print("ID to Food mapping:", id_to_food)

train_paths = [
    "/content/drive/MyDrive/foodseg103/train-00000-of-00003-6bb37ec387d1825a.parquet",
    "/content/drive/MyDrive/foodseg103/train-00001-of-00003-4a1caa37147c0681.parquet",
    "/content/drive/MyDrive/foodseg103/train-00002-of-00003-c8b698399244cd95.parquet"
]
dfs = [pd.read_parquet(p) for p in train_paths]
full_df = pd.concat(dfs, ignore_index=True)
```

Figure 22: Class Mappings and Data Loading Implementation

3.4.2 YOLO Map Generation

The Implementation seen in Figure 23 is responsible for generating the YOLO map discussed in the architecture details section. It begins by allocating an all-zero array of shape $[104, H, W]$ to hold one score map per class at the target resolution (512×512).

Next, the function loops over each bounding box and extracts the corner coordinates (x1,y1,x2,y2) and confidence score. A confidence threshold of 0.3 is declared, any bounding box with a confidence score below this is discarded to ensure the model only considers predictions with reasonable confidence. The bounding box is then scaled to fit inside the 512x512 target resolution, a small padding of 10 pixels on each side of the box is added to provide the Blend-Seg network with some additional context.

Every pixel inside the box is then multiplied by a fixed alpha value of 5. The reason for this while YOLOs confidence scores are normalised between 0 and 1, U-Nets' raw logits can be any real number positive or negative. As a result, boosting the YOLO values is necessary to ensure that when the branches are fused the YOLO information is not drowned out by the inherently larger U-Net values. Finally, in the event of overlapping bounding boxes the highest confidence score at each location is taken. The Yolo map that is returned is now a [104 x 512 x 512] array matching the layout of the U-Net logits allowing for a smooth fusion.

```
def get_yolo_map(img, target_size=(512,512), pad=10, conf_thresh=0.3, alpha=5.0, num_classes=104):
    # Prepare an empty array of shape [104, H, W] to hold one 2D score map per class
    orig_w, orig_h = img.size
    W, H = target_size
    yolo_map = np.zeros((num_classes, H, W), dtype=np.float32)
    boxes = run_yolo_once(img) # Run YOLO on the original image to get bounding boxes
    if not boxes:
        return yolo_map # If no objects detected, return the all-zero array

    scale_x = W / orig_w # Compute how much to scale box coordinates to fit the target grid
    scale_y = H / orig_h

    for box in boxes:
        coords = box.boxes[0].cpu().numpy()
        conf = float(box.conf.cpu().numpy()[0])
        if conf < conf_thresh: # Ignore boxes with low confidence
            continue

        yolo_cls = int(box.cls.cpu().numpy()[0])
        if yolo_cls not in yolo_to_unet: # Find which U-Net channel corresponds to this YOLO class
            continue
        channel = yolo_to_unet[yolo_cls] - 1

        # Scale box corners to the new resolution, then apply padding
        x1_scaled = max(0, int(coords[0] * scale_x))
        y1_scaled = max(0, int(coords[1] * scale_y))
        x2_scaled = min(W, int(coords[2] * scale_x))
        y2_scaled = min(H, int(coords[3] * scale_y))
        x1 = max(0, x1_scaled - pad)
        y1 = max(0, y1_scaled - pad)
        x2 = min(W, x2_scaled + pad)
        y2 = min(H, y2_scaled + pad)

        # set every pixel inside the box to alpha * confidence, preserving higher value if boxes overlap
        yolo_map[channel, y1:y2, x1:x2] = np.maximum(
            yolo_map[channel, y1:y2, x1:x2],
            alpha * conf
        )
    return yolo_map
```

Figure 23: YOLO Map Generation Implementation

3.4.3 Blend-Seg Architecture

The implementation of the Blend-Seg architecture is encapsulated in two small modules SimpleFusionModule and BlendSeg. These can both be seen in Figure 24

The BlendSeg module first defines the two branches. Each consisting of a 3x3 convolution followed by batch normalisation and then a ReLU activation function. These branches reduce their inputs from 104 channels down to 64.

The goal of the fusion module is to produce high-quality segmentation masks. However, U-Net’s 64-channel features already form a reasonably good segmentation mask. As a result in initial experiments, the fusion network learned to completely ignore the YOLO branch as its features were just seen as unnecessary noise. To combat this, all elements of the YOLO map are multiplied by a fixed value of 4.9 (yolo_scale) before being passed into the fusion module. This forces the network to utilise YOLO’s features to prevent the network from converging on non-optimal solutions.

The SimpleFusionModule takes in the outputs of the two branches to form a 128-channel fused feature map. This module is simply just another 3x3 convolution with batch normalisation and a ReLU activation function, followed by a 1x1 convolution that reduces 128 channels down to 104. This 104-channel output is the residual correction. At the end of the forward pass this correction is added to the initial unet_logits to produce the refined segmentation mask

```
class SimpleFusionModule(nn.Module): # Fuses the YOLO map and the U-Net logits
    def __init__(self, in_channels, num_classes):
        super(SimpleFusionModule, self).__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(in_channels, in_channels, kernel_size=3, padding=1),
            nn.BatchNorm2d(in_channels),
            nn.ReLU(inplace=True) # 3x3 convolutional layer
        )
        self.conv2 = nn.Conv2d(in_channels, num_classes, kernel_size=1)
    def forward(self, x):
        out = self.conv1(x)
        out = self.conv2(out) # 1x1 convolutional layer that reduces the 128 channels to 104 to match the U-Net logits
        return out

class BlendSeg(nn.Module):
    def __init__(self, unet_channels, yolo_channels, num_classes, fixed_yolo_scale=4.9):
        super(BlendSeg, self).__init__()
        self.num_classes = num_classes
        self.yolo_scale = fixed_yolo_scale # fixed scale factor for YOLO maps
        self.unet_branch = nn.Sequential( # defines the U-Net branch with 3x3 convolutional layer
            nn.Conv2d(unet_channels, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True) # channels reduced from 104 - 64
        )
        self.yolo_branch = nn.Sequential( # defines the YOLO branch with 3x3 convolutional layer
            nn.Conv2d(yolo_channels, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True) # channels reduced from 104 - 64
        )
        self.fusion = SimpleFusionModule(64 * 2, num_classes)

    # Forward pass through the network
    def forward(self, unet_logits, yolo_map):
        yolo_scaled = yolo_map * self.yolo_scale
        unet_feat = self.unet_branch(unet_logits)
        yolo_feat = self.yolo_branch(yolo_scaled)
        fusion_input = torch.cat([unet_feat, yolo_feat], dim=1)
        delta = self.fusion(fusion_input)
        return unet_logits + delta # residual correction
```

Figure 24: Blend-Seg Architecture Implementation

3.4.4 Loss Function

The loss function for this model seen in Figure 25, is a combination of Focal Loss and Dice Loss. Focal Loss helps the model focus on hard, misclassified pixels (Lin et al. 2017), while Dice Loss encourages good overall overlap between predicted and true masks.

Focal Loss is defined as

$$\mathcal{L}_{\text{focal}} = -(1 - p_t)^\gamma \log(p_t)$$

where p_t is the network’s predicted probability for the correct class at each pixel. The factor $(1 - p_t)^\gamma$ is called the *focusing term* (Lin et al. 2017). When the model is already confident (p_t is close to 1), this term becomes very small, so easy pixels such as the background contribute little to the loss. When the model is unsure or wrong (p_t is close to 0), it stays near 1, keeping the penalty high for hard pixels. The γ value determines how strongly to focus on the harder pixels (Lin et al. 2017). Here $\gamma = 2.0$ is used to sharply down-weight easy pixels as there is far more background pixels than foreground pixels in food segmentation tasks. This ensures learning concentrates on boundaries, small items, and overlaps.

Dice Loss is based on the Dice Similarity Coefficient (DSC) defined in Section 2.4.3

Dice Loss is defined as

$$L_{\text{dice}} = 1 - \frac{2|P \cap G| + \varepsilon}{|P| + |G| + \varepsilon}$$

where P is the set of pixels predicted as foreground, G is the set of true foreground pixels, and ε is a small constant to avoid division by zero. A perfect match gives $|P \cap G| = |P| = |G|$, making the loss zero. Dice Loss directly rewards good overall overlap, helping to correct imbalances between large background regions and smaller food items.

The final loss is simply these two loss terms added together

$$L_{\text{total}} = L_{\text{focal}} + L_{\text{dice}}.$$

This dual loss both corrects challenging local errors (via Focal Loss) and maintains accurate global shapes (via Dice Loss), resulting in sharper boundaries and more reliable overall segmentation.

```

class FocalLoss(nn.Module):
    def __init__(self, gamma=2.0, alpha=None, reduction='mean'):
        super(FocalLoss, self).__init__()
        self.gamma = gamma
        self.alpha = alpha
        self.reduction = reduction
    def forward(self, inputs, targets):
        ce_loss = nn.functional.cross_entropy(inputs, targets, reduction='none')
        pt = torch.exp(-ce_loss)
        loss = ((1 - pt) ** self.gamma) * ce_loss
        if self.alpha is not None:
            if not torch.is_tensor(self.alpha):
                self.alpha = torch.tensor(self.alpha).to(inputs.device)
            at = self.alpha[targets]
            loss = at * loss
        if self.reduction == 'mean':
            return loss.mean()
        elif self.reduction == 'sum':
            return loss.sum()
        else:
            return loss

focal_loss_fn = FocalLoss(gamma=2.0, alpha=None, reduction='mean')

def dice_loss(outputs, targets, smooth=1e-6):
    outputs = torch.softmax(outputs, dim=1)
    targets_one_hot = torch.zeros_like(outputs)
    targets_one_hot.scatter_(1, targets.unsqueeze(1), 1)
    intersection = (outputs * targets_one_hot).sum(dim=(2,3))
    union = outputs.sum(dim=(2,3)) + targets_one_hot.sum(dim=(2,3))
    dice = (2 * intersection + smooth) / (union + smooth)
    return 1 - dice.mean()

def combined_loss_fn(outputs, targets):
    focal = focal_loss_fn(outputs, targets)
    dice = dice_loss(outputs, targets)
    return focal + dice

```

Figure 25: Loss Function Implementation

3.4.5 U-Net Unfreezing

The best performing U-Net++ checkpoint trained on the FoodSeg103 dataset as part of the experiments in chapter 4 is loaded. Initially, this U-Net model is frozen for a fixed number of epochs, this value is stored in the `phase_epoch` variable seen in Figure 24. In this instance `phase_epoch` is set to 4. This means that for the first 4 epochs only the fusion layers themselves are being trained, giving the model time to learn how to merge YOLO and U-Net features without disturbing the pretrained backbone.

After 5 epochs, the optimiser and scheduler are reinitialised with the U-Net backbone unfrozen. This is done to allow the pretrained U-Net model to update its weights to better suit its new role in the unique dual-branch fusion task. In Figure 24 it can be seen that `backbone_params` has a lower learning rate than `other_params`. This setup lets the backbone adjust gently to the new fusion task, while the newly added layers continue learning more aggressively.

```

for epoch in range(num_epochs):
    # Unfreeze the backbone at phase_epoch.
    if epoch + 1 == phase_epoch:
        print("Phased fine-tuning: Unfreezing backbone (encoder) parameters.")
        for name, param in unet_model.named_parameters():
            if "encoder" in name:
                param.requires_grad = True
        # Reinitialize optimizer and scheduler with the newly unfrozen parameters.
        backbone_params = [p for n, p in unet_model.named_parameters() if p.requires_grad and "encoder" in n]
        other_params = [p for n, p in list(unet_model.named_parameters()) + list(blend_seg.named_parameters())
                        if p.requires_grad and ("encoder" not in n)]
        optimizer = optim.Adam([
            {'params': backbone_params, 'lr': 1e-5},
            {'params': other_params, 'lr': 1e-4}
        ], weight_decay=1e-5)
        scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=2, verbose=True)

```

Figure 26: U-Net Unfreezing Implementation

3.5 Evaluation and Results

After training, Blend-Seg was evaluated on the standard held-out validation set included as part of FoodSeg103. This section summarises the results and compares them to our initial U-Net model and existing state-of-the-art models.

3.5.1 Overall Validation Metrics

in Table 1. The results from the trained Blend-Seg model are compared to the original U-Net++ model that was used in the U-Net branch.

Table 1: Overall validation performance: U-Net++ baseline vs. Blend-Seg.

Evaluation Metric	U-Net++ Baseline	Blend-Seg
Pixel Accuracy	72.91%	77.01%
Mean IoU (mIoU)	17.02%	41.55%
Mean Dice	20.43%	52.93%
Food-class F1 Score	0.38	0.66

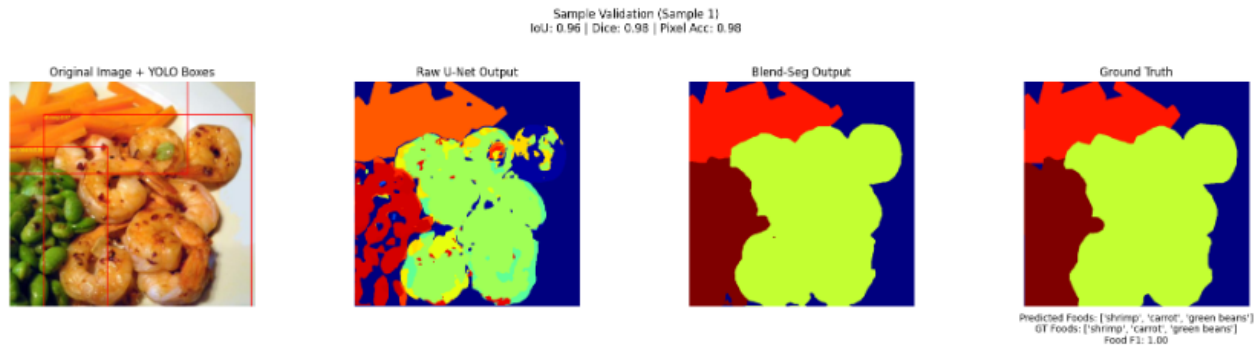
Blend-Seg achieves a large gain in both region-level overlap (mIoU and Dice) and food-class F1, demonstrating that this dual-branch blending architecture can substantially improve segmentation quality.

A full comparison between Blend-Seg and existing FoodSeg103 models is provided in Section 4.20 following the full evaluation of all experiments.

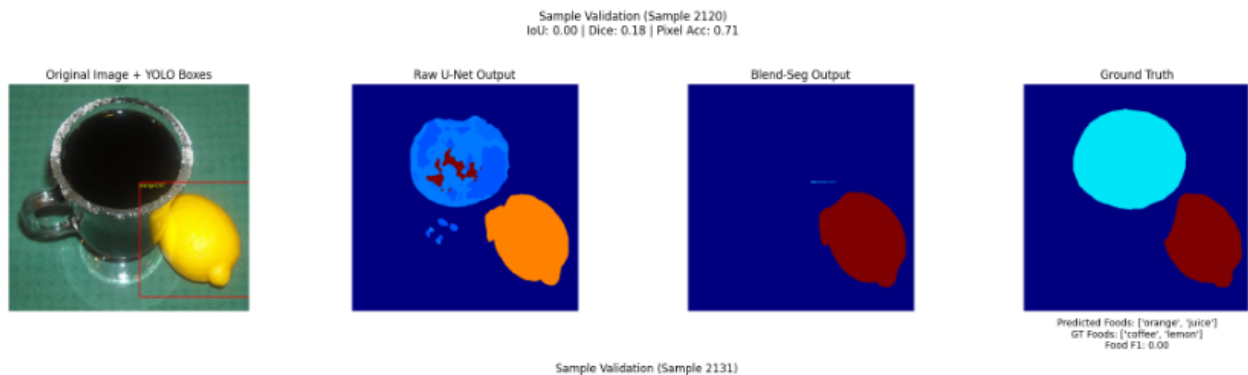
3.5.2 Visualisation of Results

Figure 27 shows one high-quality and one failure case from the validation set, highlighting where Blend-Seg excels and where further work is needed. The output for both models includes the YOLO boxes created in the YOLO branch as well as the raw U-Net output

created at the end of the U-Net branch. The ground truth mask provided as part of the FoodSeg103 dataset is included for comparison with the target result.



(a) High-quality segmentation (IoU = 0.96, Dice = 0.98, Acc = 0.98)



(b) Failure case (IoU = 0.00, Dice = 0.18, Acc = 0.71)

Figure 27: (a) Blend-Seg correctly segments shrimp, carrot, and green beans. (b) The model misses coffee and incorrectly labels the lemon.

The high-quality segmentation seen in Figure 27a highlights the improvements this approach can provide. The raw U-Net output identifies the key regions as well as some accurate boundary information, but there are scattered patches of incorrectly labelled pixels throughout them. This is what causes the poor validation results seen in Table 1. Looking at the Blend-Seg output, the improvements are obvious. By blending the YOLO and U-Net features, the final output provides a near-perfect IoU of 0.96.

The failure case seen in Figure 27b helps highlight a potential area of improvement for future variations of this architecture. The YOLO model completely misses the coffee, resulting in it being left out of the final Blend-Seg output. However, in the raw U-Net output the basic boundary of the coffee can be seen this could indicate in the present setup the U-Net features are not being weighted heavily enough, resulting in accuracy being bottlenecked by the YOLO branch. Future improvements could involve lowering the weighting of the YOLO branch allowing U-Nets localisations to be included in the final output more often.

3.6 Conclusion

The Blend-Seg prototype shows that a fusion of object-detection and dense segmentation features can deliver large practical gains on real-world food segmentation tasks. In this prototype these design choices resulted in a 24% jump in mean IoU ($17.02 \rightarrow 41.6\%$) and a 0.28 improvement in the food classification F1 score over the U-Net++ baseline, while retaining real-time inference speed. Although state-of-the-art transformer models still edge out Blend-Seg by four to five percentage points, they do so at a higher computational cost. Blend-Seg provides a balance between accuracy and efficiency, making it practical for real-world dietetic and mobile health scenarios where latency, memory, and power budgets are important.

Analysis of the visual results back this up, the fusion network consistently sharpens boundaries and removes spurious labels. However, its reliance on bounding boxes can suppress objects missed by the YOLO branch. Future iterations could explore adaptive branch weighting or confidence-aware fusion to mitigate this issue.

Overall, the prototype shows significant results for this technique.

4 Experiments

This section covers all of the experiments completed as part of this report, detailing their objective, software, hyperparameters and evaluation. The full implementation for each experiment can be found at this [GitHub](#)

4.1 Overview of the Empirical Framework

This project conducts a comprehensive series of experiments designed to evaluate and improve semantic segmentation models for food images. The experiments are divided into four groups based on model family: DeepLabV3, U-Net, YOLO and finally the proposed prototype of this report Blend-Seg. Each experiment explores a distinct aspect of model training, optimisation, or refinement. The training time of these models ranged from 3 - 18 hours. All experiments were run on Google Colab using a single T4 GPU with 15GB of VRAM. The experiments are as follows

- **Experiment 1: DeepLab:** Established a baseline DeepLab V3 model on FoodSeg103.
- **Experiment 2: DeepLab:** Applied Bayesian Optimisation to search for improved hyperparameters.
- **Experiment 3: DeepLab:** Unfroze the backbone and added new loss terms to boost performance.
- **Experiment 4: DeepLab:** Introduced stronger regularization and augmentation techniques.
- **Experiment 5: DeepLab:** Developed a simple morphological post-processing function.
- **Experiment 6: DeepLab:** Applied Dense CRF refinement to sharpen segmentation outputs.
- **Experiment 7: U-Net:** Trained a basic U-Net with a shallow ResNet-34 encoder as a baseline.
- **Experiment 8: U-Net:** Increased input resolution and switched to a ResNet-50 encoder.
- **Experiment 9: U-Net:** Upgraded to U-Net++ architecture with an EfficientNet-B4 encoder.
- **Experiment 10: YOLO:** Trained a YOLOv8 object detector using bounding boxes derived from segmentation masks.
- **Experiment 11: YOLO:** Scaled up to YOLOv8X with stronger augmentations and higher resolution.

- **Experiment 12: Blend-Seg:** Created the first prototype by boosting U-Net logits with YOLO detections.
- **Experiment 13: Blend-Seg:** Replaced the static boosting step with a lightweight learnable fusion module.
- **Experiment 14: Blend-Seg:** Developed the final Blend-Seg prototype using dual-branch feature processing and phased fine-tuning.

4.1.1 Dataset and Preparation

All experiments were conducted using the open-source FoodSeg103 dataset, which is widely used for pixel-level semantic segmentation of food images. The dataset contains 7,118 images across 104 food categories, with each image containing up to six distinct classes and associated pixel-wise masks.

The dataset was first split into a training set and a test set using an 80/20 ratio. This resulted in 3,986 training images and 997 test images. The test set was kept completely separate from the training loop to allow fair evaluation of model generalization. Only the training set was used during model training and validation stages.

4.2 Experiment 1: DeepLab

Objectives

The goal of this experiment is to establish a reproducible baseline for pixel-level food segmentation on the *FoodSeg103* dataset using DeepLab V3 with a ResNet-50 backbone. The baseline will give us an idea of early performance, expose initial problems and serve as a reference for later improvements.

Software

- **Framework:** PyTorch 2.6 running on Google Colab (single T4 GPU, 15 GB VRAM).
- **Network:** DeepLab V3 with an ImageNet-pre trained ResNet-50 backbone.
- **Loss:** CrossEntropyLoss.

The training loop implementation including data augmentation, forward pass, loss computation, and scheduler step—is shown in Fig. 28.

```

# Training + Testing
for epoch in range(HYPERPARAMS["num_epochs"]):
    model.train()
    train_loss = 0.0
    # Initialize confusion matrix for the training set.
    train_conf_matrix = np.zeros((HYPERPARAMS["num_classes"], HYPERPARAMS["num_classes"]), dtype=np.int64)

    train_bar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{HYPERPARAMS['num_epochs']} [Training]")
    for images, masks in train_bar:
        images = images.to(device)
        masks = masks.to(device)

        optimizer.zero_grad()
        if scaler is not None:
            with torch.cuda.amp.autocast():
                outputs = model(images)['out']
                loss = criterion(outputs, masks)
                scaler.scale(loss).backward()
                scaler.step(optimizer)
                scaler.update()
        else:
            outputs = model(images)['out']
            loss = criterion(outputs, masks)
            loss.backward()
            optimizer.step()

        train_loss += loss.item()

```

Figure 28: Core training loop used in Experiment 1

Hyper-parameters

The main settings are listed in Table 2. ten epochs were chosen to capture early learning dynamics without incurring long runtimes.

Table 2: Training hyper-parameters for Experiment 1

Hyper-parameter	Value	Explanation
Batch size	4	Fits within GPU memory; ensures stable updates.
Epochs	10	Enough to observe convergence trends before longer runs.
Learning rate	0.0001	Conservative step size to reduce risk of divergence.
Input size	513×513	High enough resolution to allow the model to extract important features
Flip probability	0.5	Horizontal flip for data augmentation.
Rotation degrees	$\pm 10^\circ$	Mild rotations for viewpoint invariance.

Results

Training took 3 hours on a T4 GPU Figure 29 plots the evolution of training and validation loss. Training loss declines steadily, but validation loss begins to plateau. The model was

then tested on the standard FoodSeg103 validation set which achieved the metrics in Table 3. A typical visual output showing the original image, DeepLab output and target ground truth is illustrated in Fig. 30.

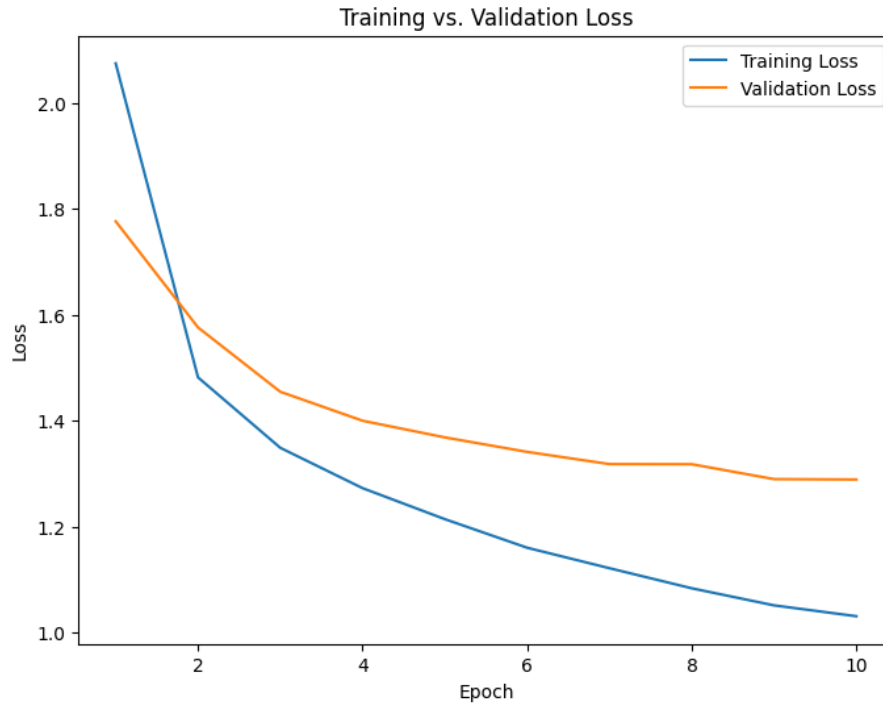


Figure 29: Training vs. validation loss for Experiment 1

Table 3: Validation metrics for Experiment 1

Metric	Score	Explanation
Pixel Accuracy	65.9 %	Measures the percentage of pixels that are correctly classified.
mIoU	10.28 %	Measures the average overlap between predicted and true regions across all classes (Intersection over Union).
mDice	12.69 %	Measures the average similarity between predicted and true regions, emphasizing region matching (Dice coefficient).
Food F1	36 %	Measures the harmonic mean of precision and recall for detecting food classes versus background.

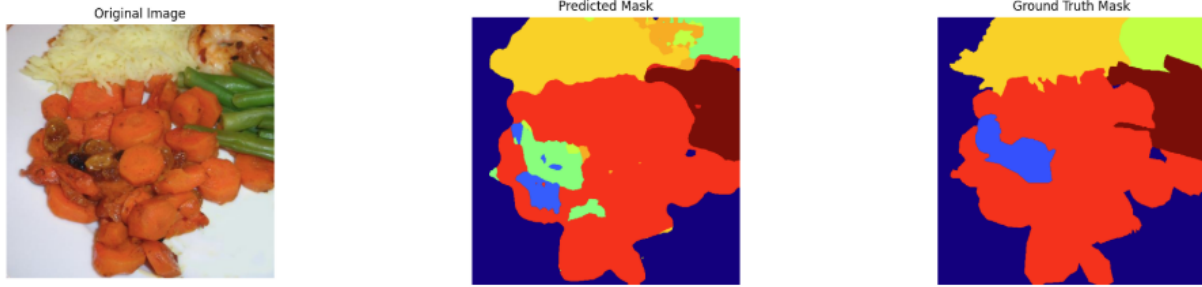


Figure 30: Visual segmentation output of Experiment 1

Evaluation

Analysis of the segmentation outputs reveals that the model may not be performing as poorly as the evaluation metrics suggest. As we can see in Figure 30 above, the model correctly identifies all of the distinct regions in the image and correctly labels the majority of the pixels within them. The performance issues seem to be mainly caused by oversegmentation, where a single food item is segmented into multiple separate regions. Exploring methods to reduce smaller regions could be a simple way of drastically improving the model’s results.

4.3 Experiment 2: DeepLab

Objectives

The goal of this experiment is to investigate whether Bayesian Optimisation (BO) can automatically discover a hyperparameter region that outperforms the manual settings from Experiment 1. Bayesian Optimisation is the process of building a probabilistic model that can be used to explore the best configuration of multiple variables. This method balances exploring uncertain areas and exploiting known high-performing regions to attempt to maximise the mIoU metric. The key question is, does the extra compute spent on BO translate into higher mIoU on the full validation set?

Software

All the software is the same as in experiment 1 with the addition of the bayes_opt BO engine. BO maximised validation mIoU by searching hyperparameters in the ranges below.

- Learning rate: $[1 \times 10^{-5}, 1 \times 10^{-3}]$
- Image scale factor: $[0.6, 1.0]$
- Flip probability: $[0, 1]$
- Rotation: $[0^\circ, 30^\circ]$

Fifteen iterations were run, each running for 6 epochs. The first 5 iterations used completely random hyperparameters to get a wide range of exploration. The next 10 iterations used to results of the 5 initial iterations to determine which parameters showed the most promise to determine what hyperparameter combinations to explore.

Running this optimiser for only 6 epochs on a small subset of the data is unlikely to lead to discovery of optimal hyperparameters, as such, the purpose of this experiment is more so to see if Bayesian Optimisation could identify promising hyperparameter regions and if its computational cost is justified for further use in future experiments.

Hyper-parameters

The best hyperparameters chosen by BO were retrained on the full training set for 10 epochs. The parameters that changed from experiment 1 can be seen in Table 4 all other hyperparameters remained the same.

Table 4: Comparison of training hyperparameters between Experiment 1 and Experiment 2

Hyper-parameter	Experiment 1	Experiment 2
Learning rate	0.0001	0.00073
Input size	513×513	503×503
Flip probability	0.5	0.3745
Rotation degrees	$\pm 10^\circ$	$\pm 18^\circ$

Results

Training took 3 hours, the test vs train graph seen in Figure 31 shows an even larger gap between the train and test losses than those seen in experiment 1. Validation scores are contrasted directly with the baseline in Table 5. Finally, a representative segmentation mask is shown in Fig. 32.



Figure 31: Training vs. validation loss for Experiment 2

Table 5: Side-by-side validation metrics for Experiments 1 and 2

Metric	Exp. 1	Exp. 2
Pixel Accuracy	65.9 %	64.11 %
mIoU	10.28 %	9.46 %
mDice	12.69 %	11.71 %
Food F1	36 %	34 %

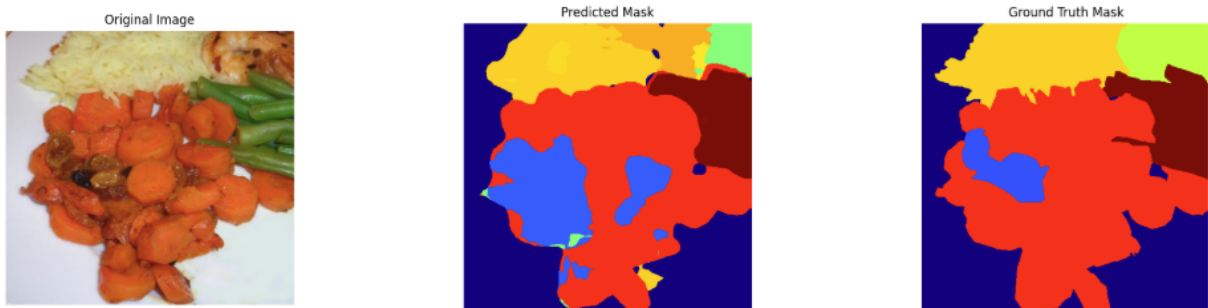


Figure 32: Visual segmentation output of Experiment 2

Evaluation

Every metric in Table 5 drops marginally relative to the baseline, and Fig. 31 shows a gap between training and validation loss. The likely culprit is the limited BO search budget. Restricting each iteration to six epochs did not allow enough time for practical improvements to be found. Additionally, the BO optimisation search took 3 hours to run, which gives the initial impression that the small increase gained by finding the truly optimal hyperparameters may not be worth the computational cost.

While not demonstrating good results for this experiment, the concept of Bayesian Optimisation is still worth exploring further. In further experiments, the number of iterations, epochs and training data will all be increased to get a better idea of the improvements the process can provide under more optimal circumstances.

4.4 Experiment 3: DeepLab

Objectives

As a continuation from Experiment 2, the Bayesian Optimisation process was extended to enlarge the search scope. The idea was to let BO explore a wider region by increasing the number of samples from 150 to 700, as well as the number of epochs from 6 to 10. On top of that, partial backbone unfreezing and extra loss terms were introduced to improve the model’s ability to handle complex multi-ingredient images. The broader goal is to see whether these two changes richer losses and trainable high-level features translate into a tangible bump in mIoU.

Software

One of the key changes to the training process was the introduction of backbone unfreezing, which lets the model fine-tune higher-level feature extractors. The amount of the backbone that is unfrozen is controlled with the `unfreeze_frac` hyperparameter.

Another key change was the addition of two more loss functions, these being classification loss and boundary loss. Classification loss encourages the model to match its overall predictions to the correct set of food items in the image. This should help address the oversegmentation issue.

Boundary loss applies a harsher penalty to the predicted edges of a region in order to refine sharp object boundaries. With these new loss terms, the total loss becomes cross entropy + classification loss + dice loss + boundary loss. Hyperparameters were added to control the weighting for each of these loss functions, these values were then chosen by the Bayesian Optimisation.

All other implementation remains the same. Figure 33 shows the code snippet that applies partial unfreezing and wires in the new losses.

```

# -----
# Build Model
# -----
model = torch.hub.load('pytorch/vision:v0.10.0', 'deeplabv3_resnet50', pretrained=True)
# Initially freeze the backbone.
for param in model.backbone.parameters():
    param.requires_grad = False
# Unfreeze a fraction of backbone layers.
unfreeze_frac = OPT_HYPERPARAMS["unfreeze_frac"]
children = list(model.backbone.children())
n_children = len(children)
n_unfreeze = int(unfreeze_frac * n_children)
print(f"Unfreezing last {n_unfreeze} layers out of {n_children} in backbone.")
if n_unfreeze > 0:
    for child in children[-n_unfreeze:]:
        for param in child.parameters():
            param.requires_grad = True

model.classifier[4] = nn.Conv2d(256, OPT_HYPERPARAMS["num_classes"], kernel_size=1)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# -----
# Loss, Optimizer, Scheduler, and AMP Scaler
# -----
criterion = nn.CrossEntropyLoss()
dice_weight = OPT_HYPERPARAMS["dice_weight"]
cls_loss_weight = OPT_HYPERPARAMS["cls_loss_weight"]
boundary_loss_weight = OPT_HYPERPARAMS["boundary_loss_weight"]
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()),
                        lr=OPT_HYPERPARAMS["learning_rate"],
                        weight_decay=OPT_HYPERPARAMS["weight_decay"])
scheduler = ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=1, verbose=True)
scaler = torch.cuda.amp.GradScaler() if OPT_HYPERPARAMS["use_amp"] and device.type == 'cuda' else None
# -----

```

Figure 33: Implementation snippet for Experiment 3

Hyper-parameters

Training length was increased to 25 epochs to better visualise the model’s improvements over time. Key differences from Experiment 2 are shown in Table 6.

Table 6: Comparison of hyperparameters between Experiment 2 and Experiment 3

Hyper-parameter	Experiment 2	Experiment 3
Learning Rate	0.00073	0.0006216
Weight Decay	0.000156	0.0001919
Dice Weight	N/A	0.9919
Classification Loss Weight	N/A	0.4121
Boundary Loss Weight	N/A	0.1577
Unfreeze Fraction	Frozen	0.2024

Results

The full training run took roughly 12 hours to complete on the T4. The validation loss curve in Figure 34 begins to stall after epoch 10, whereas training loss keeps falling. The 25-epoch

run yielded the metrics in Table 7. For context, the same results from experiments 1 and 2 are repeated.

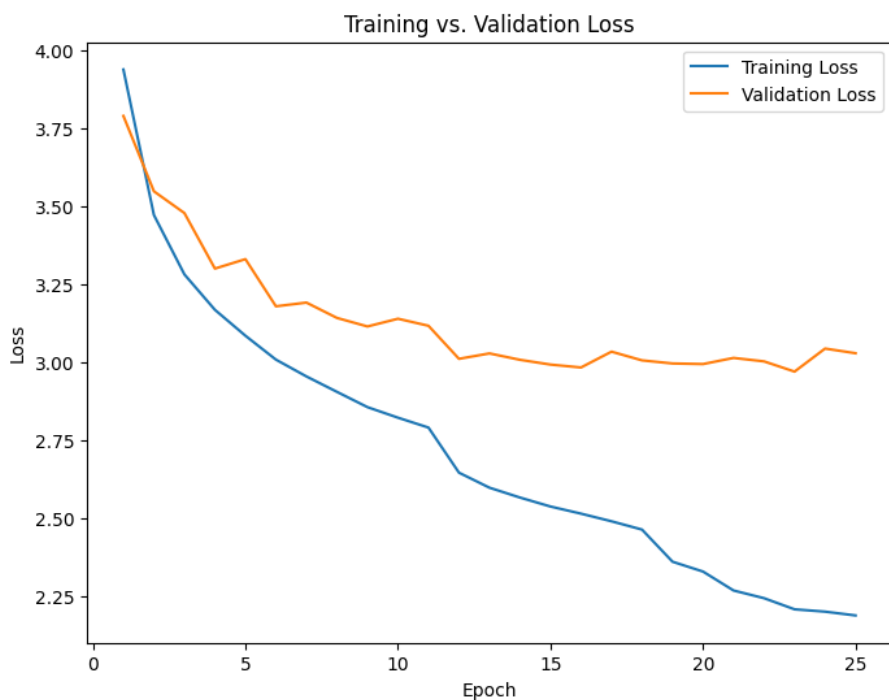


Figure 34: Training vs. validation loss for Experiment 3

Table 7: Validation metrics for Experiment 1 + 2 + 3

Metric	Exp. 1	Exp. 2	Exp. 3
Pixel Accuracy	65.9%	64.11 %	71.76 %
mIoU	10.28%	9.46 %	15.98 %
mDice	12.69%	11.71 %	18.62 %
Food F1	36%	34 %	42 %

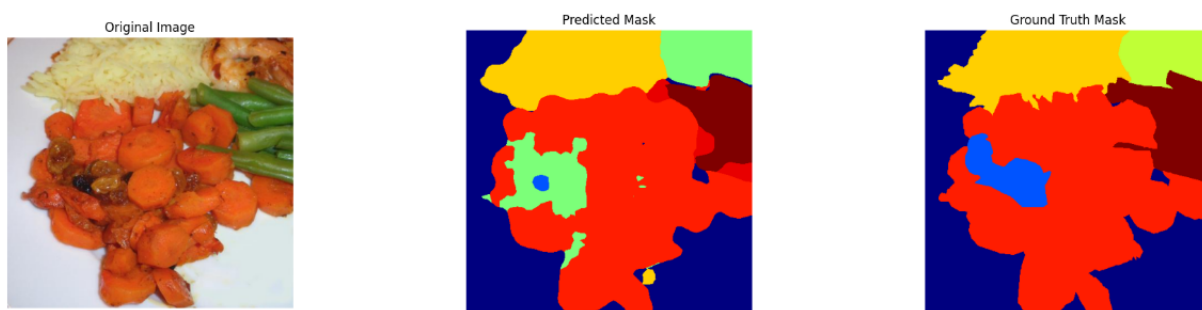


Figure 35: Segmentation output of Experiment 3

Evaluation

Experiment 3 displays superior results in every metric than the previous experiments. The added classification and boundary losses help the network identify distinct food items more accurately and produce cleaner boundaries. Partial unfreezing contributes too: allowing 20% of the backbone to adapt to FoodSeg103 improves the model’s ability to capture important features. The jump to 25 epochs did not provide much improvement Figure. 34 shows that after 10 epochs, the model’s ability to learn stalls. Future work will focus on dropout, colour jitter, and other augmentations in an attempt to keep the validation loss improving over more epochs.

Finally, the jump from three-hour training times to twelve-hour training times shows the trade-off of these deeper techniques. Accounting for the increased epochs, training slowed from 3.3 epochs an hour to 2.08 epochs an hour. Subsequent experiments will need to consider the relationship between potential metric gains and increased training times.

4.5 Experiment 4: DeepLab

Objectives

The goal of this experiment is to investigate whether extended regularisation and data augmentation techniques can allow the validation loss to continue improving for longer.

Software

Regularisation was strengthened in this experiment by introducing dropout and colour jittering augmentations.

Hyper-parameters

Table 8 compares the key hyperparameter changes introduced relative to Experiment 3. The batch size was increased to better utilise the GPU and reduce noise in the training data. Learning rate was increased at the same rate as per the linear scaling rule. All other settings remained the same as in Experiment 3.

Table 8: Comparison of key hyperparameters between Experiment 3 and Experiment 4

Hyper-parameter	Experiment 3	Experiment 4
Epochs	25	30
Dropout Rate	None	0.3
Colour Jitter Brightness	None	0.2
Colour Jitter Contrast	None	0.2
Colour Jitter Saturation	None	0.2
Colour Jitter Hue	None	0.1
Learning Rate	0.0006216	0.0012432
Batch Size	4	8

Results

Training took approximately 14 hours to complete. Figure 36 shows the training and validation loss curves, while Table 9 summarises validation metrics compared across all previous experiments.

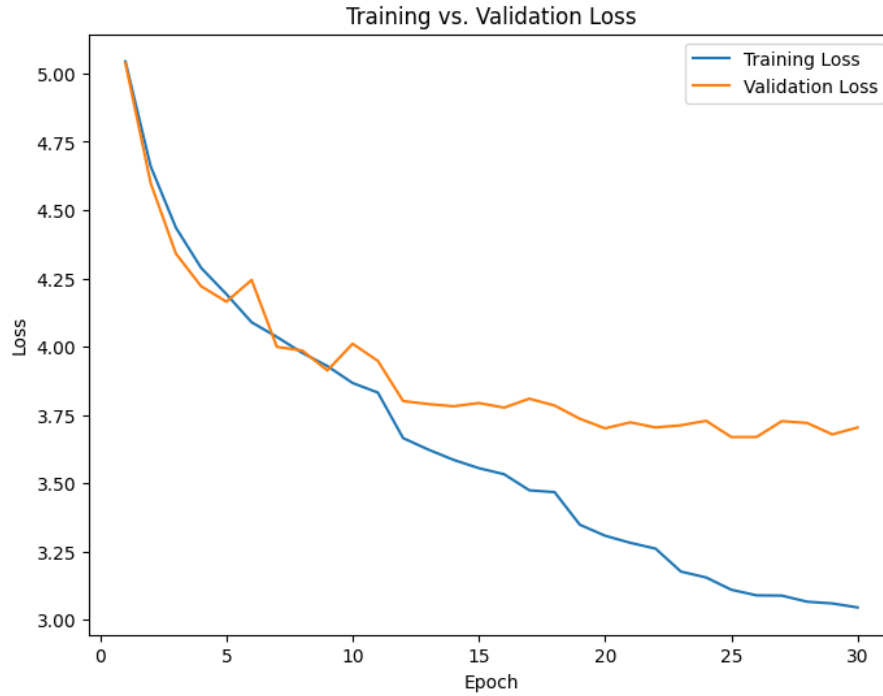


Figure 36: Training vs. validation loss for Experiment 4

Table 9: Side-by-side validation metrics for Experiments 1–4

Evaluation Metric	Exp. 1	Exp. 2	Exp. 3	Exp. 4
Pixel Accuracy	65.9%	64.11%	71.76%	67.18%
mIoU	10.28%	9.46%	15.98%	15.44%
mDice	12.69%	11.71%	18.62%	19.06%
Food F1	36%	34%	42%	46%

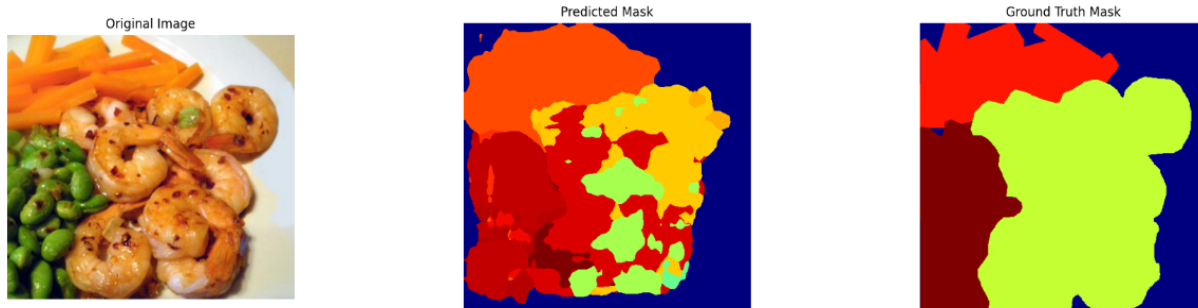


Figure 37: Visual segmentation output of Experiment 4

Evaluation

Experiment 4 introduced extended regularisation and data augmentation techniques into the DeepLab V3 training loop, but these additions failed to meaningfully close the generalisation gap. Although Dice and Food F1 scores improved slightly compared to Experiment 3, Pixel Accuracy and mIoU declined, suggesting that the additional augmentations added noise without substantially improving the model’s generalisation ability.

Importantly, looking at Figure 37, segmentation quality visually degrades despite comparable metric scores to previous experiments. This suggests that although the model achieved its highest Food F1 score so far, this came at the cost of finer boundary precision. As the network prioritised classifying the correct food types more strongly, its ability to draw accurate region outlines weakened. Future experiments will focus on ways to boost classification consistency without sacrificing segmentation detail.

4.6 Experiment 5: DeepLab

Objectives

While previous experiments focused on improving the training process, this experiment focus on a different approach. The goal of this experiment is to investigate whether a simple post-processing step applied to the raw segmentation masks can reduce over-segmentation issues that persisted throughout previous experiments.

Software

A custom post-processing function was developed to refine the segmentation masks. This function removes small disconnected regions, applies morphological smoothing operations, and merges fragmented regions of the same class label if they lie within a distance threshold. Table 10 summarises the key post-processing steps introduced. This post-processing function was applied directly to the model trained in experiment 3.

Table 10: Summary of post-processing operations introduced in Experiment 5

Step	Description
Small-Region Removal	Eliminates connected regions below a minimum pixel threshold to reduce fragmented predictions.
Morphological Smoothing	Applies opening and closing operations to clean noisy boundaries.
Label Cleanup	Merges nearby regions of the same class if they fall within a predefined distance threshold.

Results

Post-processing was applied to the validation set outputs from Experiment 3. Figure 38 illustrates two examples: a successful post-processed mask and a failure case where excessive merging removed legitimate boundaries. Table 11 compares the model’s validation metrics before and after post-processing.

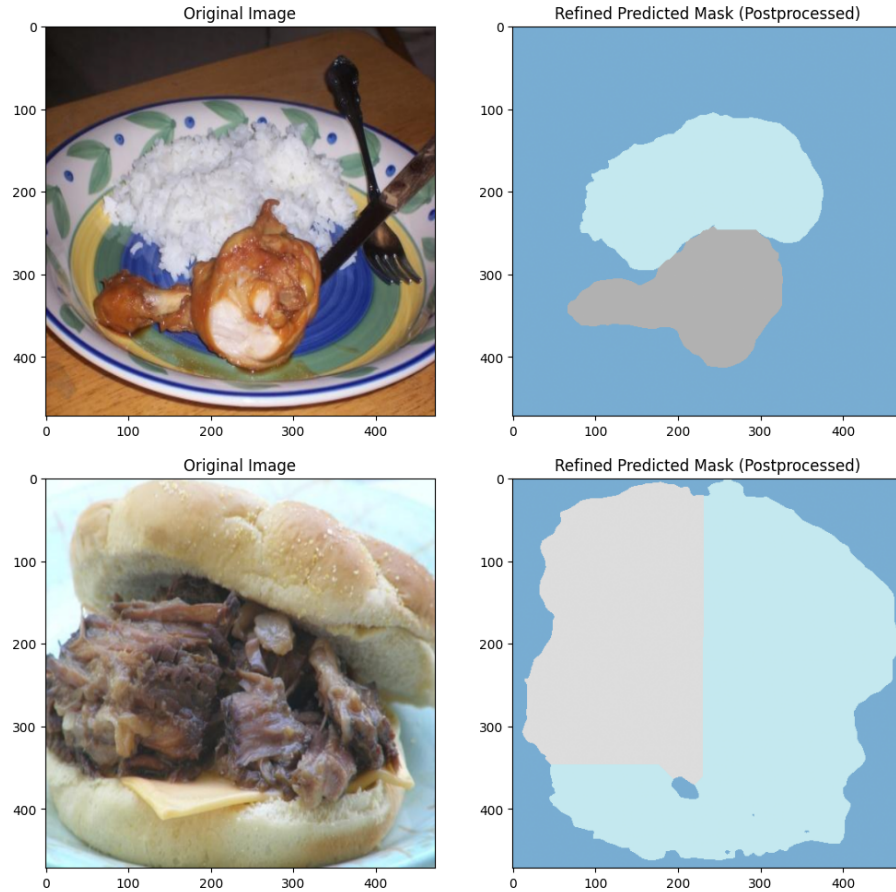


Figure 38: Examples of refined predicted masks (top) and a poor outcome with excessive merging (bottom)

Table 11: Comparison of validation metrics before and after post-processing

Evaluation Metric	Raw Output (Exp. 3)	+ Post-processing
Pixel Accuracy	71.76%	65.27%
mIoU	15.98%	19.13%
mDice	18.26%	23.53%
Food F1	42%	47.23%

Evaluation

Experiment 5 demonstrates that simple post-processing can significantly improve segmentation performance. As shown in Table 11, mIoU and mDice scores increased substantially, and Food F1 improved by over 5%. However, Pixel Accuracy declined, likely because small background regions were removed or merged incorrectly, as seen in the failure case of Figure 38.

Overall, the trade-off appears favourable: improved region consistency and reduced fragmentation outweigh occasional merging errors. Future work will focus on improving these post-processing techniques.

4.7 Experiment 6: DeepLab

Objectives

While Experiment 5 explored simple morphological post-processing to reduce over-segmentation, this experiment focuses on a more sophisticated refinement technique. The goal is to investigate whether applying a Dense Conditional Random Field (CRF) to the raw outputs of the model trained in Experiment 3 can further improve segmentation quality.

Software

A Dense CRF post-processing function was implemented using the `pydensecrf` library. This function refines the pixel-wise softmax probability outputs by incorporating both local appearance and spatial smoothness cues. Table 12 summarises the CRF operations introduced. The CRF was applied directly to the validation set outputs from Experiment 3 without any retraining of the model.

Table 12: Summary of CRF refinement operations introduced in Experiment 6

Step	Description
Softmax to Unary Conversion	Transforms the model’s class probabilities into negative log-likelihood unary potentials for the CRF.
Gaussian Pairwise Potential	Encourages spatially close pixels to share the same label to promote smoothness.
Bilateral Pairwise Potential	Encourages pixels with similar colour and proximity to share labels, helping preserve object edges.
Mean-Field Inference	Runs 10 iterations of approximate inference to compute the refined segmentation output.

Results

CRF refinement was applied to the validation set outputs from Experiment 3. Table 13 compares the validation metrics of the raw DeepLab model, the metrics with basic post-processing and the metrics with CRF refinement. Figure 39 illustrates two examples: one showing successful boundary refinement and another showing a failure case where fine details were over-smoothed.

Table 13: Comparison of validation metrics before and after basic and CRF post-processing

Evaluation Metric	Raw Output (Exp. 3)	+ Basic Post-processing	+ CRF Post-processing
Pixel Accuracy	71.76%	65.27%	73.17%
mIoU	15.98%	19.13%	30.94%
mDice	18.26%	23.53%	34.51%
Food F1	42%	47.23%	57.94%

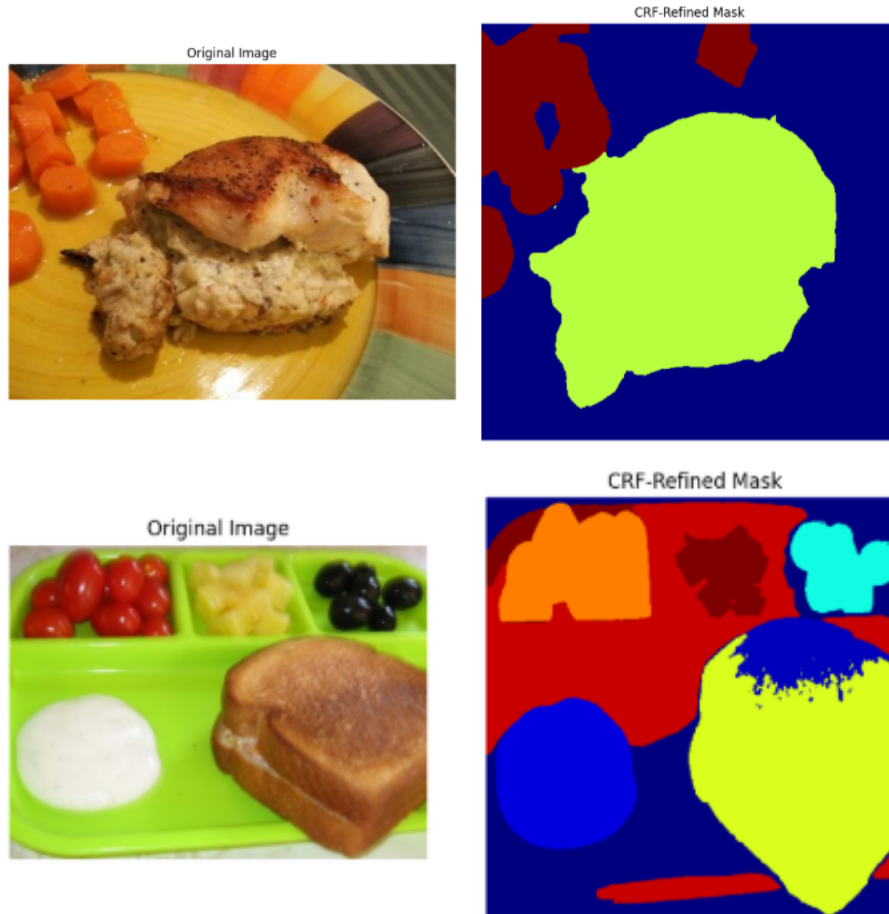


Figure 39: Examples of CRF-refined predicted masks (top) and a poor outcome with over-smoothing (bottom)

Evaluation

The addition of CRF causes both mIoU and mDice metrics to nearly double while Food F1 sees a 16% increase. This shows that this technique helps resolve the oversegmentation problem while still maintaining pixel-level segmentation information.

Although there are still some problems with this technique, in the bottom sample of Figure 39, the CRF leads large sections of the bowl to be classified as an object. Similarly, in the top results while the overall segmentation is accurate, the CRF caused an orange section of the plate to be falsely identified as a carrot due to its proximity to correctly identified carrot pixels.

Another important point to mention is the large increase in inference time that CRF brings about. It took 3 hours to run through the full validation set when applying CRF. For comparison, the same model without CRF refinement ran through the validation set in 8 minutes. This means DeepLab + CRF as currently implemented runs 22.5 times slower than just DeepLab on its own. A key goal moving forward will be to speed up this inference time

while maintaining the high-quality results.

The results of this experiment were the catalyst that inspired the creation of Blend-Seg. The original idea was to use YOLO as additional context to further guide the post-processing function to further improve these promising results while speeding up inference time.

4.8 Overall Evaluation of DeepLab Experiments

This series of experiments systematically explored methods to improve pixel-level segmentation performance using DeepLab V3 on the FoodSeg103 dataset. Starting from a baseline setup, successive experiments investigated hyperparameter optimization, richer loss functions, stronger regularization, and post-processing refinements. Each stage aimed to either boost segmentation accuracy, sharpen boundary quality, or address over-segmentation challenges while keeping computational cost in mind.

The final validation results for each experiment are summarized in Table 14 below.

Table 14: Validation results across all six DeepLab experiments

Experiment	Description	Pixel Accuracy	mIoU	mDice	Food F1
Exp. 1	Baseline DeepLab V3	65.9%	10.28%	12.69%	36%
Exp. 2	Bayesian Optimized Hyperparameters	64.11%	9.46%	11.71%	34%
Exp. 3	Backbone Unfreezing + Extra Losses	71.76%	15.98%	18.62%	42%
Exp. 4	Extended Regularization	67.18%	15.44%	19.06%	46%
Exp. 5	Basic Post-processing	65.27%	19.13%	23.53%	47.23%
Exp. 6	CRF Refinement	73.17%	30.94%	34.51%	57.94%

Looking across the experiments, several important trends emerge. The introduction of backbone unfreezing and richer loss terms in Experiment 3 led to the first major improvement over the baseline, with mIoU increasing from 10.28% to 15.98%. However, even with additional regularization techniques such as dropout and colour jitter introduced in Experiment 4 the validation loss continued to plateau early, suggesting that training process refinements alone could only bring limited gains.

Post-processing strategies proved to be a powerful complement to model improvements. Experiment 5 demonstrated that simple morphological operations could substantially boost mIoU and mDice by reducing fragmented regions, even if Pixel Accuracy suffered slightly. Experiment 6, which applied Dense CRF refinement, achieved dramatic improvements across all metrics, nearly tripling the baseline mIoU and sharply boosting Food F1. However,

the cost of this was a $22.5\times$ increase in inference time, highlighting the tension between segmentation quality and computational efficiency.

These insights helped guide the decisions made while optimising the U-Net model in the next phase of experiments.

4.9 Experiment 7: U-Net

Objectives

The goal of this experiment is to establish a baseline performance for the U-Net architecture on the FoodSeg103 dataset. Here we assess how a straightforward cross-entropy plus Dice-loss setup performs before introducing more advanced techniques.

Software

The U-Net model was implemented in PyTorch using the `segmentation_models_pytorch` library for the backbone, and `Albumentations` for data augmentation. A ResNet-34 encoder pretrained on ImageNet was used. Training employed a combination of class-weighted cross-entropy loss and Dice loss to help address the heavy class imbalance present in FoodSeg103.

Hyper-parameters

Table 15 lists the key hyperparameters used. The image size was kept relatively small at 256×256 pixels to balance GPU memory usage and input resolution. All settings were selected to provide a fast initial benchmark rather than optimal performance.

Table 15: Key hyper-parameters for Experiment 7

Hyper-parameter	Value
Batch Size	24
Number of Epochs	20
Learning Rate	0.0001
Weight Decay	1×10^{-5}
Image Size	256×256 pixels
Optimizer	Adam
Scheduler	ReduceLROnPlateau (patience=3)
Augmentations	Resize, Rotate, Brightness (all probability 0.5)

Results

Training took approximately 3 hours to complete. Figure 40 shows the training and validation loss curves. Table 16 summarises the best validation metrics achieved during the run, and a typical segmentation output is displayed in Figure 41.

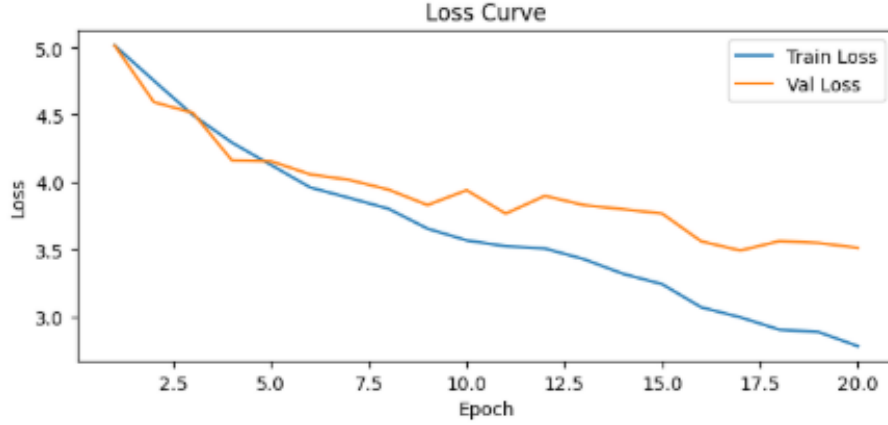


Figure 40: Training vs. validation loss for Experiment 7

Table 16: Validation metrics for U-Net Experiment 7

Evaluation Metric	Best Validation Result
Pixel Accuracy	32.39%
mIoU	1.39%
mDice	1.82%
Food F1	11%

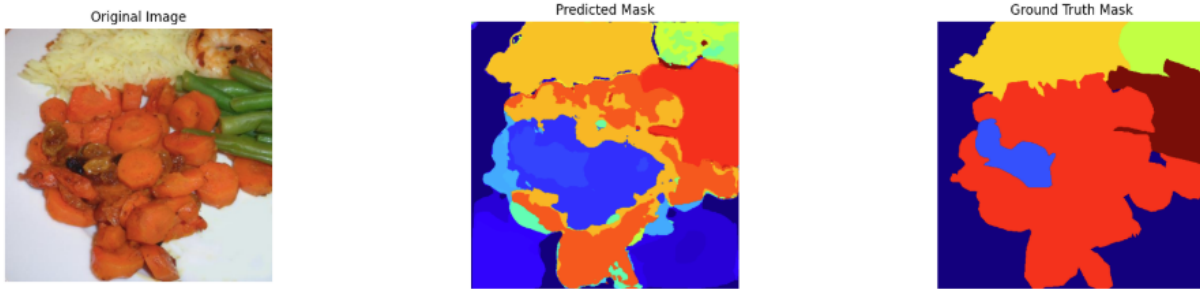


Figure 41: Visual segmentation output of U-Net Experiment 7

Evaluation

Experiment 7 establishes that a basic U-Net with a shallow ResNet-34 encoder and low-resolution inputs performs poorly on the highly multi-class FoodSeg103 dataset. As seen in Table 16, all metrics remain extremely low, with mIoU and mDice below 2%, and only 11% Food F1 score. Figure 41 further confirms that the model struggles to produce coherent segmentation masks.

The small image size of 256×256 and the limited capacity of the encoder were major limiting factors. Future experiments will focus on increasing input resolution, deepening the encoder, and adjusting loss functions to better capture the fine-grained structures required for accurate food segmentation.

4.10 Experiment 8: U-Net

Objectives

Building on the baseline U-Net from Experiment 7, this experiment explores whether a deeper encoder and improved loss functions can boost segmentation performance. In particular, the encoder was upgraded from ResNet-34 to ResNet-50, and the loss function was changed from cross-entropy to Focal Loss to better handle the heavily imbalanced FoodSeg103 training data.

Software

The U-Net model was again implemented using `segmentation_models_pytorch`, with data augmentations handled by `Albumentations`. A ResNet-50 encoder pretrained on ImageNet replaced the shallower ResNet-34. Focal Loss was combined with Dice Loss to help the model focus on hard-to-classify pixels and encourage better region overlap.

Hyper-parameters

Table 17 lists the key hyperparameters used in this experiment. Compared to Experiment 7, the image resolution was increased, and the batch size was reduced to accommodate the deeper backbone and larger input size within GPU memory limits.

Table 17: Comparison of hyperparameters between Experiments 7 and 8

Hyper-parameter	Experiment 7	Experiment 8
Encoder Backbone	ResNet-34 (ImageNet)	ResNet-50 (ImageNet)
Number of Epochs	20	30
Image Size	256×256 pixels	512×512 pixels
Batch Size	24	6
Loss Function	Cross-Entropy + Dice	Focal + Dice

Results

Training took approximately 14 hours to complete. Figure 42 shows the training and validation loss curves; the train loss starts off extremely large before quickly dropping, but both train and validation loss continue to decrease throughout training. Table 18 summarises the best validation metrics achieved. A typical segmentation output is displayed in Figure 43.

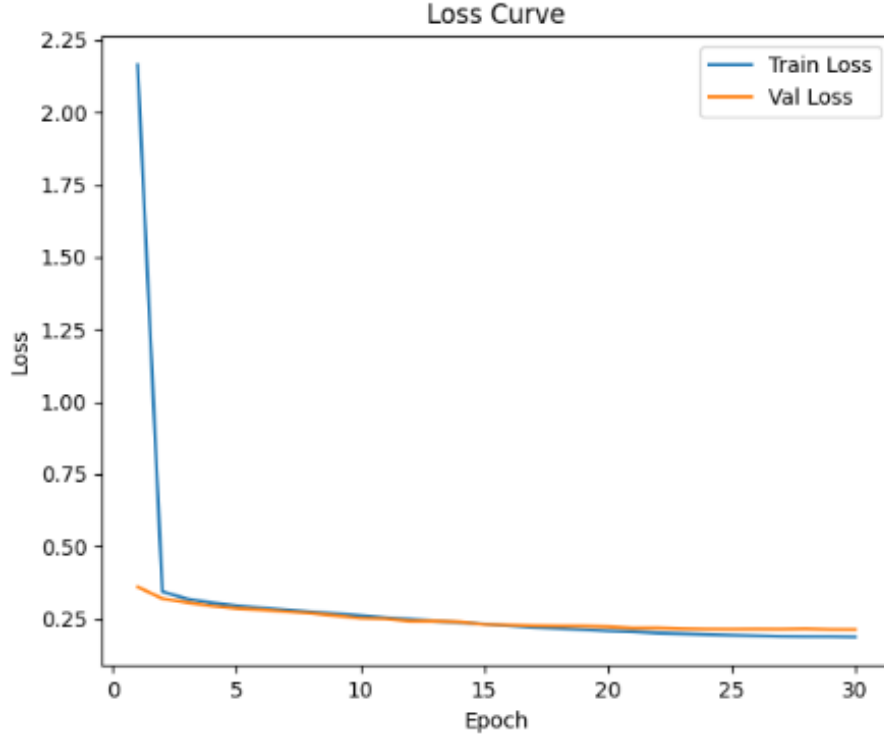


Figure 42: Training vs. validation loss for U-Net Experiment 8

Table 18: Side-by-side validation metrics for Experiments 7 and 8

Evaluation Metric	Experiment 7	Experiment 8
Pixel Accuracy	32.39%	72.07%
mIoU	1.39%	15.25%
mDice	1.82%	18.52%
Food F1	11%	34%

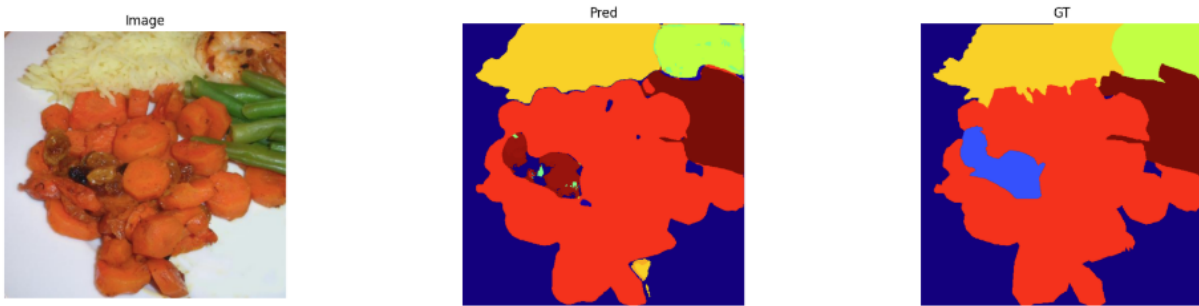


Figure 43: Visual segmentation output of U-Net Experiment 8

Evaluation

The results shown in Table 18, demonstrate large improvements in Pixel Accuracy, mIoU, mDice, and Food F1 compared to Experiment 7. Figure 43 shows that the model now captures food regions much more cleanly, although some fine-grained errors remain.

The use of a deeper ResNet-50 encoder and higher-resolution inputs helped the model extract richer features, while Focal Loss improved its ability to handle rare classes. These results confirm that increasing backbone capacity and tailoring loss functions are critical steps toward achieving higher-quality food segmentation.

4.11 Experiment 9: U-Net

Objectives

Building on the improvements seen in Experiment 8, this experiment explores whether further deepening the model architecture can continue to improve segmentation performance. The architecture is upgraded from standard U-Net to U-Net++, which replaces direct skip connections with nested dense convolutional blocks to enable richer multi-scale feature fusion (Zhou et al. 2018). Additionally, the encoder is upgraded from ResNet-50 to the more powerful EfficientNet-B4 pretrained on ImageNet.

Software

The model was implemented using `segmentation_models_pytorch`, with U-Net++ selected as the core architecture. The ResNet-50 encoder from Experiment 8 was replaced with EfficientNet-B4 to provide deeper and more efficient feature extraction. All other training configurations, including the Focal + Dice loss combination, remained the same.

Hyper-parameters

Table 19 compares the key hyperparameter changes introduced relative to Experiment 8. The number of epochs was kept constant at 30 to allow a fair comparison of improvements across experiments.

Table 19: Comparison of key hyper-parameters between Experiments 8 and 9

Hyper-parameter	Experiment 8	Experiment 9
Encoder Backbone	ResNet-50 (ImageNet)	EfficientNet-B4 (ImageNet)
Architecture	U-Net	U-Net++
Number of Epochs	30	30

Results

Training took approximately 15 hours to complete. Figure 44 shows the training and validation loss curves, while Table 20 summarises the best validation metrics achieved.

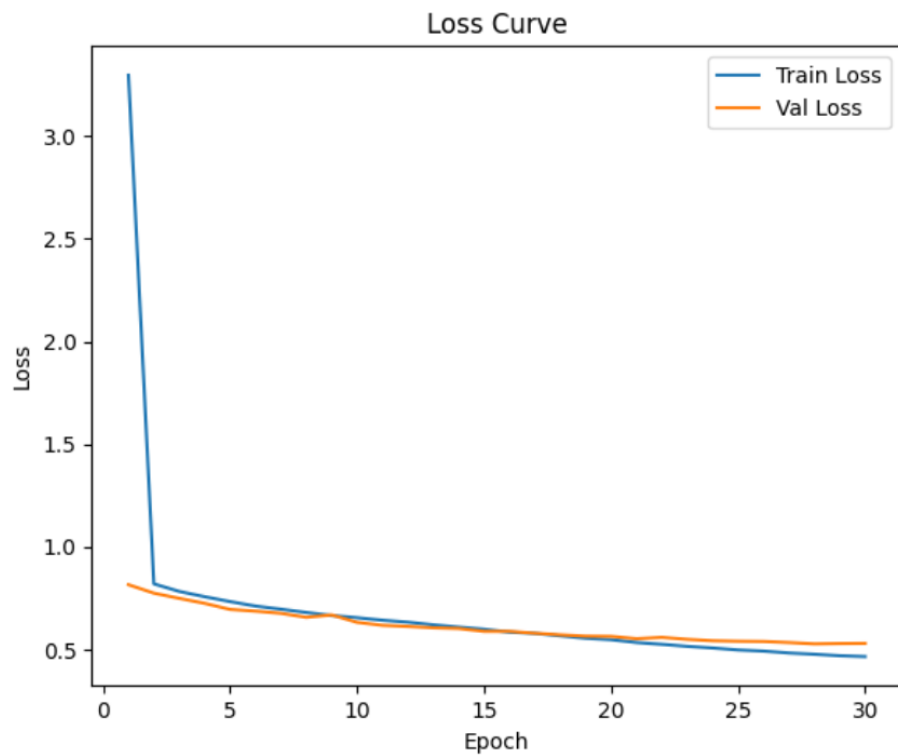


Figure 44: Training vs. validation loss for U-Net Experiment 9

Table 20: Side-by-side validation metrics for U-Net Experiments 7, 8, and 9

Evaluation Metric	Experiment 7	Experiment 8	Experiment 9
Pixel Accuracy	32.39%	72.07%	72.91%
mIoU	1.39%	15.25%	17.02%
mDice	1.82%	18.52%	20.43%
Food F1	11%	34%	38%

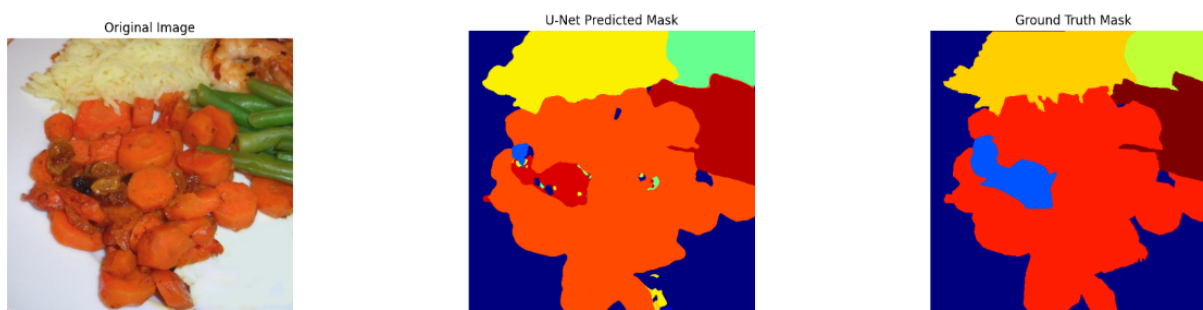


Figure 45: Visual segmentation output of U-Net Experiment 9

Evaluation

Experiment 9 demonstrates modest gains over Experiment 8. As shown in Table 20, each evaluation metric improved by 1–4%. The output in Figure 45 further confirms that while overall segmentation quality has improved slightly, the differences are subtle compared to the jump seen from Experiments 7 to 8.

The results suggest diminishing returns from simply deepening the encoder or switching to a more complex architecture without changing the overall learning approach.

4.12 Overall Evaluation of U-Net Experiments

The U-Net experiments explored varied methods to improve pixel-level segmentation performance. Starting from a basic U-Net with a ResNet-34 encoder, successive experiments investigated deeper backbones, richer loss functions, and more advanced architectures such as U-Net++. Each stage aimed to enhance feature extraction, better handle class imbalance, and improve fine-grained segmentation.

No experiment testing CRF refinement on U-Net’s performance was conducted because the Blend-Seg prototype was designed to replace the post-processing techniques explored in Experiments 5 and 6. Therefore, only the raw pre-processed results were considered when selecting the architecture for the prototype. If time had permitted, post-processing techniques would also have been applied to U-Net for a more complete comparison with DeepLab

Table 21: Validation results across all U-Net experiments

Experiment	Description	Pixel Accuracy	mIoU	mDice	Food F1
Exp. 7	Baseline U-Net (ResNet-34)	32.39%	1.39%	1.82%	11%
Exp. 8	Deeper U-Net (ResNet-50)	72.07%	15.25%	18.52%	34%
Exp. 9	U-Net++ (EfficientNet-B4)	72.91%	17.02%	20.43%	38%

Results showed that deeper backbones significantly improved segmentation quality. Moving from ResNet-34 to ResNet-50 boosted mIoU by over 13%, while U-Net++ with EfficientNet-B4 yielded modest further gains. However, improvements plateaued compared to the large jump seen between the baseline and deeper U-Net, highlighting that backbone depth and architecture alone could not fully overcome the challenges of food segmentation.

4.13 Comparison of U-Net VS DeepLab

The best performing U-Net++ model (Experiment 9) and the best raw DeepLab V3 model (Experiment 3) are compared side-by-side in Table 22.

Table 22: Comparison of best raw DeepLab and U-Net++ models

Evaluation Metric	DeepLab V3 (Exp 3)	U-Net++ (Exp 9)
Pixel Accuracy	71.76%	72.91%
Mean IoU (mIoU)	15.98%	17.02%
Mean Dice	18.62%	20.43%
Food F1 Score	42%	38%

As shown in Table 22, the raw DeepLab V3 and U-Net++ models achieve broadly comparable results on FoodSeg103. U-Net++ slightly outperforms DeepLab in terms of Pixel Accuracy, Mean IoU, and Mean Dice, while DeepLab maintains a marginally higher Food F1 score.

After examining these results, U-Net++ was selected as the base segmenter for the Blend-Seg prototype for several reasons. First, U-Net++ produced smoother and more spatially coherent raw outputs, making it easier to combine with object detection signals. Second, U-Net++ offered a simpler and more stable training process, requiring fewer tuning adjustments compared to DeepLab’s more sensitive hyperparameters. Finally, the small advantage DeepLab has in Food F1 score is less relevant in the proposed Blend-Seg architecture, where the YOLO branch will be responsible for the bulk of the classification.

4.14 Experiment 10: YOLO

Objectives

This experiment focuses on building an object detection branch for the Blend-Seg dual-branch prototype. Specifically, a YOLO v8 model is trained on FoodSeg103, using bounding box annotations derived from semantic segmentation masks. The goal is to establish a strong object detection baseline capable of localising multiple fine-grained food classes.

Dataset Preparation

The FoodSeg103 dataset was reused, but pixel masks were converted into bounding box annotations. Connected regions containing more than 10 pixels of the same class were extracted. The topmost, bottommost, leftmost, and rightmost pixel coordinates defined each bounding box, with an additional 10-pixel padding applied to preserve context. A `food_yolo.yaml` configuration file listing class names and dataset paths was created. The dataset retained the same 80/20 train-validation split used in previous experiments.

Software

Training was conducted using the Ultralytics v8.3.6 framework on Google Colab. The YOLOv8-large (YOLOv8l) model variant was used. Strong data augmentation settings were enabled, and loss computation and evaluation metrics were handled internally by the Ultralytics framework.

Hyper-parameters

Key training hyperparameters are summarised in Table 23. Rare-class oversampling was applied to improve model performance on under-represented food items.

Table 23: Training hyper-parameters for Experiment 10

Hyper-parameter	Value	Notes
Batch Size	8	Maximum size fitting GPU memory.
Image Size	640×640	Recommended input size for YOLOv8l models.
Number of Epochs	70	Sufficient time to ensure full convergence.
Data Augmentation	Mosaic 1.0, MixUp 0.1, RandAugment, Erasing 0.4	Strong augmentation strategy; disabled after epoch 10.
Rare-class Oversampling	$\times 3$ duplication	Helps the model learn rare, under-represented food classes.

Results

Training took approximately 4.5 hours to complete. Figure 46 shows the validation box loss, classification loss, and distribution-focused loss (dfl_loss) during training. All three losses steadily decrease, stabilising after around 40 epochs. A slight up-tick in classification loss after epoch 50 suggests overfitting in the classification head, but overall losses remain stable.

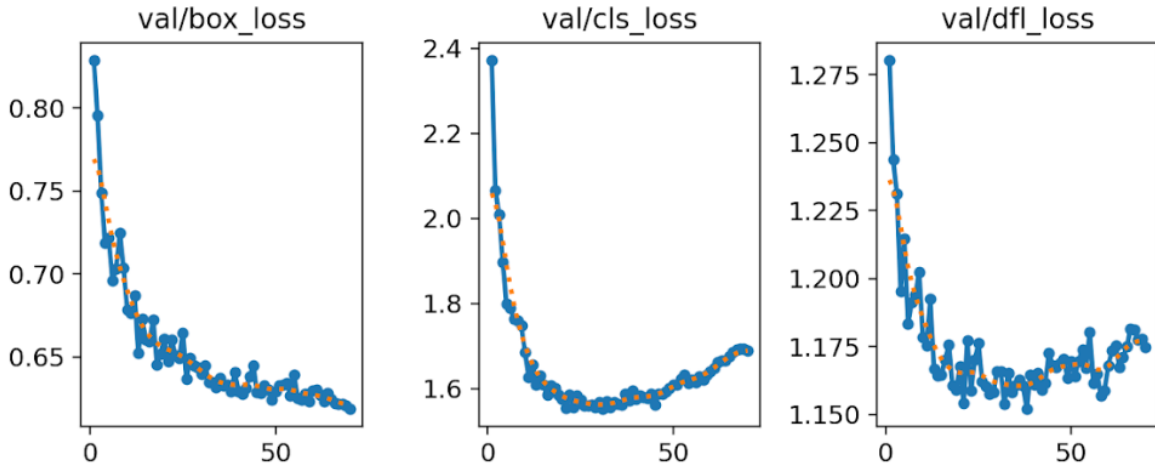


Figure 46: Validation box, classification, and distribution-focused losses during training.

YOLO uses object detection metrics rather than pixel-level segmentation metrics. The main metrics are:

- **Precision (P):** Percentage of predicted boxes that are correct ($\text{IoU} \geq 0.5$).
- **Recall (R):** Percentage of ground-truth objects successfully detected.
- **mAP₅₀:** Mean Average Precision at IoU 0.5; a standard object detection benchmark.

- **mAP_{50:95}**: Mean AP averaged over IoU thresholds at 0.50, 0.55, ..., 0.95 (harder metric).
- **Inference Time**: Time to process a single image during validation.

Table 24 reports the aggregate validation scores at the best checkpoint (epoch 45). Figure 47 shows sample detection outputs on validation images. Boxes are reasonably tight, and confidence scores are high for major food categories.

Table 24: Validation metrics for Experiment 10

Evaluation Metric	Value
Precision (P)	55%
Recall (R)	36%
mAP ₅₀	38.7%
mAP _{50:95}	32.9%



Figure 47: Sample detection outputs from Experiment 10 on validation images.

Evaluation

Experiment 10 successfully establishes a strong object detection baseline on FoodSeg103. As shown in Table 24, the model achieved an mAP₅₀ of 38.7% and an mAP_{50:95} of 32.9%, with a precision of 55% and recall of 36%. These results are promising given the difficulty of detecting over 100 fine-grained food classes.

Visual inspection of detection outputs (Figure 47) shows good localisation for major food items but some missing detections for smaller or occluded objects.

4.15 Experiment 11: YOLO

Objectives

This experiment investigates whether upgrading the detection model to YOLOv8x, increasing image resolution, and introducing stronger augmentations can boost detection performance. The goal is to build a more powerful object detection branch for the Blend-Seg prototype that better handles the complex, fine-grained classes found in FoodSeg103.

Software

Training was performed using the Ultralytics v8.3.98 framework on Google Colab. The YOLOv8x model variant was selected for its deeper backbone. Advanced data augmentations including Mosaic, MixUp, AutoAugment, and Albumentations were enabled to enrich the training set.

Hyper-parameters

Key hyperparameters are summarised in Table 25. Compared to Experiment 10, image size was increased from 640×640 to 800×800 , batch size was adjusted to fit GPU memory, and multi-scale training was activated.

Table 25: Training hyper-parameters for Experiment 11

Hyper-parameter	Value	Notes
Model	YOLOv8x	Deeper and wider backbone for better feature extraction.
Batch Size	12	Fits within A100 GPU memory for 800×800 inputs.
Image Size	800×800	Higher resolution to capture small object details.
Number of Epochs	60	Extended training to allow full convergence.
Data Augmentation	Mosaic 1.0, MixUp 0.1, AutoAugment	Strong augmentation strategy; mosaic closed after epoch 10.
Rare-class Oversampling	$\times 3$ duplication	Improves detection of rare food classes.
Multi-scale Training	True	Image sizes randomly scaled between 640–1024.

Results

Training took approximately 2.7 hours to complete. Figure 48 shows the validation box loss, classification loss, and distribution-focused loss (dfl_loss) curves during training. Losses steadily decrease, although the classification loss shows a mild upward drift after epoch 50, suggesting early overfitting. Table 26 directly compares the best validation metrics achieved in Experiment 11 against those from Experiment 10. Figure 49 shows sample detection outputs on validation images.

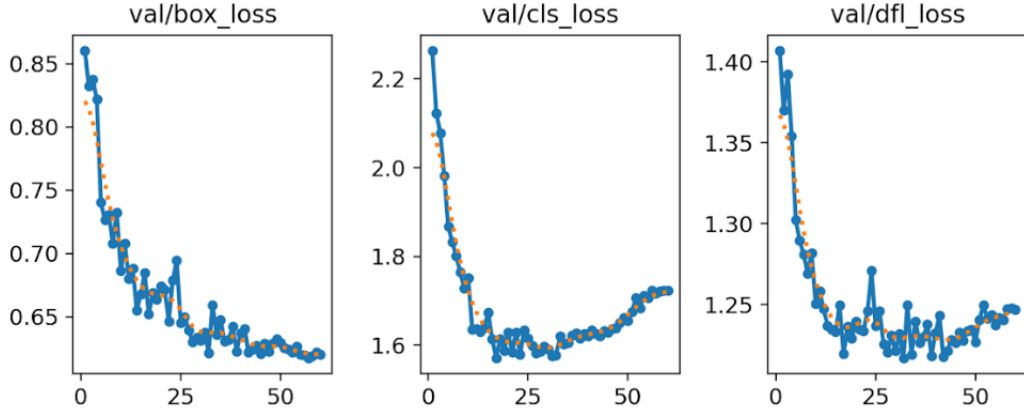


Figure 48: Validation box, classification, and distribution-focused losses for Experiment 11.

Table 26: Side-by-side validation metrics for Experiments 10 and 11

Evaluation Metric	Exp. 10 (YOLOv8l)	Exp. 11 (YOLOv8x)
Precision (P)	55.0%	54.8%
Recall (R)	36.0%	36.0%
mAP ₅₀	38.7%	39.4%
mAP _{50:95}	32.9%	33.7%



Figure 49: Sample detection outputs from Experiment 11.

Evaluation

Experiment 11 produced modest improvements over Experiment 10. As seen in Table 26, mAP_{50} improved by +0.7%, $\text{mAP}_{50:95}$ by +0.8%, with a trade-off of -0.2% precision. The more powerful model does not seem to scale notably better to the dataset. This YOLOv8x model is the best-performing of the two experiments and is used in the following Blend-Seg prototypes

4.16 Experiment 12: Blend-Seg

Objectives

The initial idea was for Blend-Seg to be a post-processing technique which improved upon those used in DeepLab experiments 5 and 6. The goal was to refine U-Nets output by creating a post-process function that uses YOLO to provide additional classification and location context.

Software

The synergy pipeline was implemented in PyTorch, using U-Net (from `segmentation_models_pytorch`) for pixel segmentation and YOLOv8 (via the Ultralytics library) for bounding box detection.

The Blend-Seg synergy pipeline consists of the following steps:

- Run U-Net across the full resized image (512×512) to obtain per-pixel logits.
- Run YOLOv8l on the original image to detect bounding boxes and class scores.
- For each high-confidence bounding box (≥ 0.3 confidence), boost the corresponding U-Net logit channel inside the bounding box area by an offset proportional to confidence (scaling factor $\alpha = 3$).
- Argmax over all classes to produce the final blended segmentation mask.

The process is quite simple, essentially YOLOs confidence scores are scaled and added to the U-Net logits boosting U-Net’s confidence in classes YOLO detects in that region.

Results

Processing the full FoodSeg103 validation set took approximately 17 minutes on a T4 GPU. Results are summarised below

Table 27: Validation metrics for Blend-Seg Experiment 1 and DeepLab+CRF for comparison

Evaluation Metric	Blend-Seg (Exp. 1)	DeepLab+CRF
Pixel Accuracy	76.07%	73.17%
mIoU	33.65%	30.94%
mDice	37.20%	34.51%
Food F1	58.87%	57.94%
Validation Time	17 Minutes	3 Hours

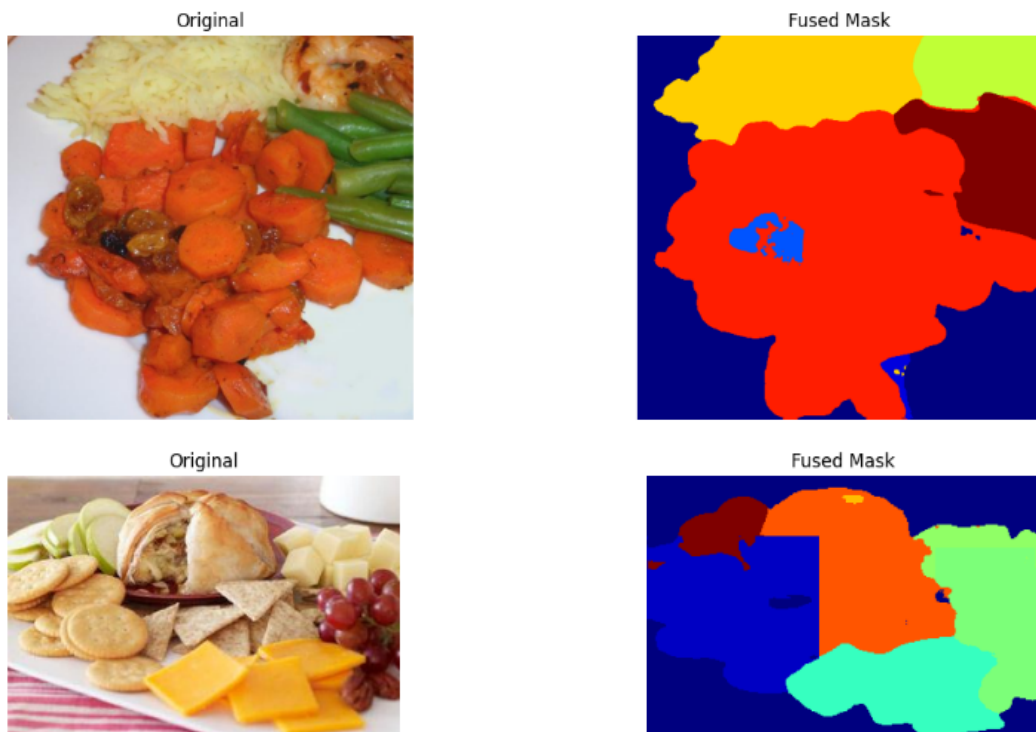


Figure 50: Examples of high-quality Blend-Seg output (top) and a poor outcome with excessive merging (bottom)

Evaluation

The first Blend-Seg prototype continues to build on the results achieved from the CRF post-processing. Table 27 shows increases in all metrics over DeepLab experiment 6 with significant jumps of roughly 3% in both mIoU and mDice. In addition to this the validation set was processed in only 17 minutes as opposed to the 3 hours taken by the CRF implementation. This makes Blend-Seg both more accurate and over 17 times faster than our previous best pipeline.

Many of the outputs of this prototype are very high quality like the one seen on the top of

Figure 50 with IoU values greater than 90%. However, this implementation has 1 key point of failure. Since this is a simple post-processing function the YOLO confidence scores are increased by the same fixed scaling factor for every sample. This is a problem because for images where U-Nets predictions are less confident like the messy overlapping image seen in the bottom output of Figure 50 the YOLO labels end up entirely overpowering the U-Net logits. This results in harsh bounding box shaped segmentation masks. The only way to fix this would be to lower the fixed YOLO scaling factor, which does result in increased performance on more difficult images, but for the simpler pictures where U-Net is more confident a lower scaling factor means YOLOs impact is too small to meaningfully impact the U-Net logits.

There is no one size fits all approach to choosing the YOLO scaling factor which means we must find a way where the impact of the YOLO model can scale dynamically depending upon the image being processed. To solve this problem, a lightweight fusion network was implemented which could learn how to best combine the YOLO confidence scores with the U-Net logits. This fusion network is the key component of the final Blend-Seg prototype and is the focus of the next two experiments.

4.17 Experiment 13: Blend-Seg

Objectives

The goal of this experiment was to improve upon the original Blend-Seg prototype by making the YOLO influence dynamically learnable. Rather than applying a fixed scaling factor to the YOLO confidence scores, a lightweight fusion module was trained to learn how much trust to place in YOLO detections relative to the U-Net logits at each spatial location. The hypothesis was that learning this fusion process end-to-end would produce better segmentation masks than the fixed-ratio blending used in Experiment 12.

Software

This prototype was built by integrating the pretrained U-Net++ and YOLOv8X models trained in previous experiments. A fusion module was introduced to combine the U-Net logits with a single-channel YOLO confidence map. The fusion module consisted of a small neural network that took the U-Net logits and YOLO map as inputs and learned to dynamically adjust the influence of YOLO’s object-level predictions at each spatial location. Specifically, instead of statically boosting YOLO predictions by a fixed scaling factor, the fusion module produced a learned, spatially-varying weighting mask that controlled how much trust to place in YOLO detections versus the original U-Net outputs. This allowed the model to automatically adapt the blending strength depending on the content of the image

Hyper-parameters

Key training settings for this experiment are shown in Table 28. Compared to the original setup, the biggest change was the replacement of the static YOLO scaling step with a fully learnable scaling parameter inside Blend-Seg.

Table 28: Key hyper-parameters for Blend-Seg Experiment 13

Hyper-parameter	Value
U-Net Encoder	EfficientNet-B4 (pretrained)
YOLO Backbone	YOLOv8X
Input Size	512×512 pixels
Batch Size	4
Number of Epochs	30
Learning Rate	0.0001
Weight Decay	1×10^{-5}
Loss Function	Focal Loss + Dice Loss
Data Augmentation	Horizontal Flip (p=0.5)

Results

Training took approximately 6 hours to complete. The results are summarised below

Table 29: Validation metrics comparison between U-Net++ baseline and Blend-Seg experiments

Evaluation Metric	U-Net++ Baseline	Blend-Seg Exp 12	Blend-Seg Exp 13
Pixel Accuracy	72.91%	76.07%	71.4%
Mean IoU (mIoU)	17.02%	33.65%	18.68%
Mean Dice	20.43%	37.20%	20.21%
Food F1 Score	38%	58.87%	41%

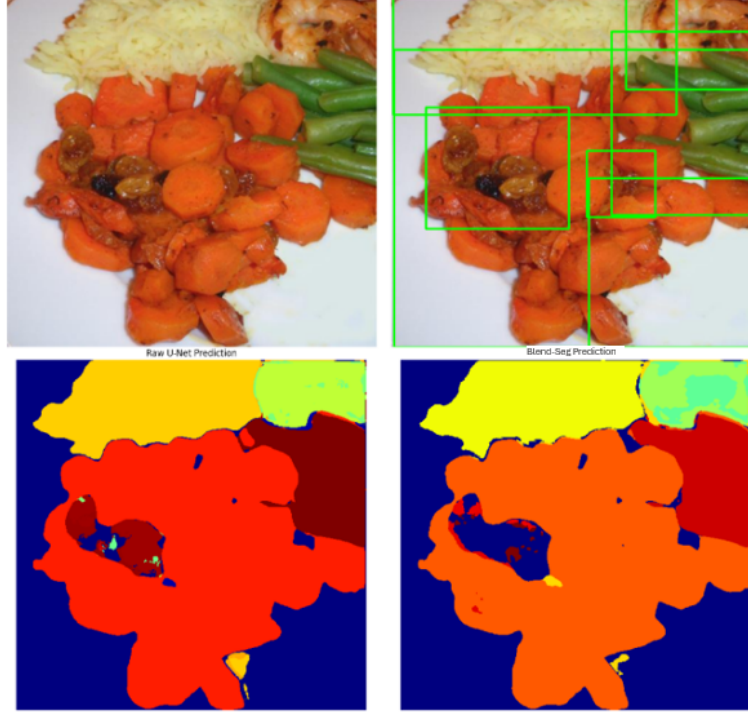


Figure 51: Visual segmentation output of Blend-Seg Experiment 13

Evaluation

While the motivation behind this experiment was promising, the results were ultimately disappointing. As seen in Table 29, Mean IoU and Mean Dice scores remained low, only slightly improving over the baseline U-Net++ model without approaching the performance achieved by the original Blend-Seg prototype.

Visual inspection of the outputs in Figure 51 reveals that the model struggled to properly balance U-Net and YOLO features. In most cases, the refined output remained highly similar to the raw U-Net output, suggesting that the fusion module was unable to meaningfully fuse the two sources of information.

These results highlight that simply learning a dynamic scaling factor is insufficient. A more sophisticated dual-branch design capable of more intelligently fusing the features of the two models, is needed to unlock the full potential of combined object and pixel-level information. Blend-Seg Experiment 14 addresses these limitations by introducing a deeper and more structured fusion pipeline, as detailed in the next section.

4.18 Experiment 14: Blend-Seg

Objectives

This experiment implements the final Blend-Seg prototype described in Chapter 3. The goal was to build a complete dual-branch segmentation system where the U-Net++ log-

its and the YOLO-derived maps are processed separately through lightweight convolutional branches and then fused via a dedicated fusion module. Unlike previous experiments, this model fine-tunes both the U-Net++ backbone and the Blend-Seg head simultaneously, aiming to maximise segmentation quality by fully leveraging both pixel-level and object-level information.

Software

Detailed coverage on the implementation of this experiment is covered in Chapter 3

Hyper-parameters

Table 30: Key hyper-parameters for Blend-Seg Experiment 14

Hyper-parameter	Value
U-Net Encoder	EfficientNet-B4 (pretrained)
YOLO Backbone	YOLOv8X
Input Size	512×512 pixels
Batch Size	3
Number of Epochs	60
Learning Rate	1×10^{-4} (non-backbone), 1×10^{-5} (backbone after epoch 5)
Weight Decay	1×10^{-5}
Loss Function	Focal Loss + Dice Loss
Data Augmentation	Horizontal Flip (p=0.5)
Phased Fine-Tuning	Backbone unfrozen after 5 epochs

Results

Training took approximately 18 hours to complete. The results are summarised below

Table 31: Validation metrics comparison between U-Net++ baseline and Blend-Seg Experiments 12, 13, and 14

Evaluation Metric	U-Net++ Baseline	Blend-Seg Exp 12	Blend-Seg Exp 13	Blend-Seg Exp 14
Pixel Accuracy	72.91%	76.07%	71.4%	77.01%
Mean IoU (mIoU)	17.02%	33.65%	18.68%	41.55%
Mean Dice	20.43%	37.20%	20.21%	52.93%
Food F1 Score	38%	58.87%	41%	66%

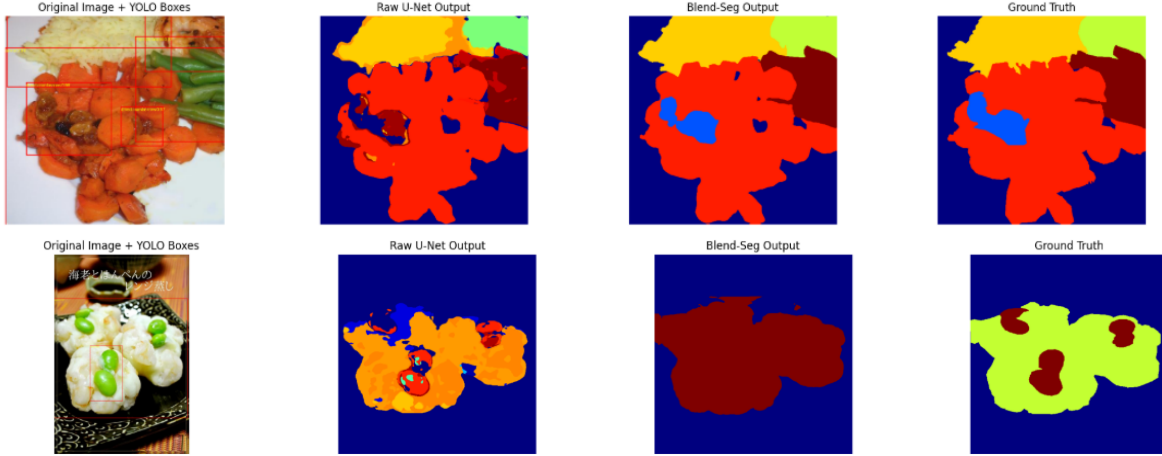


Figure 52: Examples of high-quality Blend-Seg Experiment 14 output (top) and a poor outcome (bottom)

Evaluation

This experiment demonstrates major improvements over all previous experiments. As shown in Table 31, the final prototype achieved a Mean IoU of 41.55%, Mean Dice of 52.93%, and a Food F1 score of 66% representing a 24.5% increase in mIoU original U-Net++ baseline.

Visual inspection of the outputs (Figure 52) shows that the model accurately captures both coarse regions and fine boundaries, a challenge that had persisted throughout earlier experiments. The poor result at the bottom of Figure 52 shows how the YOLO branch can sometimes overpower the segmentation details which is one of the architecture’s biggest weaknesses.

Overall, Blend-Seg Experiment 14 represents the most successful implementation developed in this project and establishes a strong foundation for future real-world deployment.

4.19 Final Evaluation of Experiments

This section summarises the best segmentation models developed throughout the course of this project. Models based on DeepLab V3, U-Net++, and the final Blend-Seg dual-branch prototypes are compared side-by-side. Table 32 presents the highest validation results achieved for each major experiment family.

Table 32: Summary of best validation results across all major experiments

Model	Pixel Accuracy	Mean IoU (mIoU)	Mean Dice	Food F1 Score
DeepLab V3 (Exp 3)	71.76%	15.98%	18.62%	42%
U-Net++ (Exp 9)	72.91%	17.02%	20.43%	38%
DeepLab V3 + CRF (Exp 6)	73.17%	30.94%	34.51%	57.94%
Blend-Seg Post-Process (Exp 12)	76.07%	33.65%	37.20%	58.87%
Blend-Seg Final Prototype (Exp 14)	77.01%	41.55%	52.93%	66%

The progression shown in Table 32 highlights the major improvements achieved over the course of this project. Early experiments using DeepLab V3 and U-Net++ established strong pixel-level segmentation baselines but struggled to fully capture the complex structure of food images. The introduction of post-processing techniques such as Dense CRF (DeepLab Experiment 6) produced significant improvements, nearly doubling Mean IoU over the baseline DeepLab models at the cost of heavily increased inference time.

However, the largest performance gains came from the development of the Blend-Seg synergy framework. By combining fine-grained pixel segmentation with object detection context, Blend-Seg achieved dramatic improvements in every evaluation metric. The final prototype achieved a Mean IoU of 41.55%, a Mean Dice of 52.93%, and a Food F1 score of 66%, representing the highest overall segmentation quality attained during this work.

These experimental findings validate the effectiveness of blending object-level and pixel-level information for complex food segmentation and lay a strong foundation for future research in this area.

4.20 Comparison to Existing Models

To provide more context to these results Table 33 compares Blend-Segs results to those of the top-performing models on the FoodSeg103 dataset.

Table 33: mIoU comparison against existing FoodSeg103 models.

Model	mIoU	Reference
FPN (ResNet-50)	27.8%	Wu et al. 2021
PVT-S	31.3%	Wu et al. 2021
DeepLabV3+	34.2%	Wu et al. 2021
CCNet (ResNet-50)	35.5%	Papers With Code 2025
Blend-Seg (Ours)	41.6%	Custom Model
SeTR-MLA (ViT-16/B)	45.1%	Wu et al. 2021
FoodSAM	46.4%	Papers With Code 2025

This table demonstrates that Blend-Seg outperforms all of the traditional CNN based approaches (FPN, PVT-S DeepLabV3+, CCNet). While recent transformer-based methods (SeTR-MLA at 45.1% and FoodSAM at 46.4%) achieve slightly higher overlap scores, they depend on large-scale ViT backbones and extensive pretraining, which dramatically increase both computational cost and inference latency (Yanyu Li et al. 2022). In contrast, Blend-Seg’s dual-branch design—fusing a lightweight YOLO head with a U-Net++ decoder—delivers competitive accuracy with far fewer parameters, faster training, and real-time inference capability. This makes Blend-Seg particularly well suited for practical deployment in dietetics applications where efficiency and responsiveness are paramount (Karabay, Varol, and Chan 2025).

4.21 Threats to Validity

While this project demonstrates substantial improvements in segmentation performance, several factors may limit the generalisability and robustness of the results. First, all experiments were conducted solely on the FoodSeg103 dataset. Although FoodSeg103 is a large and challenging dataset, it may not fully represent the diversity of real-world food imagery. Testing on additional datasets would be necessary to confirm that the improvements achieved by Blend-Seg generalise across different types of food, imaging conditions, and cultural cuisines. Only using one dataset also meant that hyperparameter tuning was primarily guided by validation set performance. This introduces a potential bias towards models that perform well on this specific validation split rather than the underlying segmentation task more generally.

Addressing these limitations by validating on multiple datasets would strengthen the claims made by this project and provide a more robust assessment of the Blend-Seg approach.

5 Discussion and Conclusion

This project set out to explore the current landscape of semantic segmentation models in the context of food image analysis. The work started with a systematic literature review, providing a solid overview of convolutional neural networks, segmentation architectures like U-Net and DeepLab, and the unique challenges presented by food image datasets such as FoodSeg103. With this foundation, a series of experiments were conducted to evaluate the performance of both U-Net and DeepLab under a variety of training conditions, exploring everything from simple baselines to more advanced setups involving Bayesian optimisation, multi-loss functions, and partial backbone unfreezing. Alongside this, a YOLO model was trained for object detection on the same dataset, with bounding box information later used to enhance segmentation performance. The culmination of this work was the design and implementation of Blend-Seg, a novel dual-branch architecture that combines YOLO’s object detection capabilities with U-Net’s pixel-level segmentation accuracy. Blend-Seg demonstrated a 24.5% boost in mIoU over my baseline U-Net model.

The results achieved during this project are not just a technical success, they are a step toward solving a real-world problem. There is a large market for AI-powered dietary apps, for example, in 2023, MyFitnessPal generated \$310 million in revenue while boasting 200 million users (Curry 2025). However, even state-of-the-art models can not consistently produce high-quality segmentation results, with none achieving more than 50% mIoU on the benchmark Food-Seg103 dataset. (Wu et al. 2021). The Blend-Seg architecture introduced in this paper provides a measurable boost in performance over traditional CNN architectures. These initial results suggest that fusing complementary model types can lead to genuinely better outputs. Importantly, Blend-Seg does this without relying on heavyweight transformer-based architectures, which are often too slow and resource-hungry for real-time applications (Yanyu Li et al. 2022). Blend-Seg offers a practical middle ground by providing better accuracy than traditional models, while remaining lightweight and deployable. This kind of balance is crucial for real-world use cases like a diet-tracking app.

Of course, there’s plenty of room for future development. The key limiting factor in my prototype is the U-Net model, which only managed to achieve a mIoU of 17.02%. Improving upon this with a better optimised semantic segmentation model could result in a large boost to the results. Another key area for improvement lies in the weighting between the YOLO and U-Net branches within the Blend-Seg architecture. In some failure cases, over-reliance on YOLO caused important items to be missed entirely. Adaptive fusion strategies, where the model dynamically adjusts how much it trusts each branch depending on the context could mitigate this. From a training standpoint, stronger regularisation and data augmentation techniques may help combat the overfitting observed in some of the experiments. Finally, expanding this work to additional datasets could give a better indication of how this technique truly stacks up to existing architectures. Overall, while there is still ground to be covered, this report provides strong evidence that this architecture offers a promising and practical alternative to traditional segmentation models, particularly in real-world dietetics applications where efficiency and accuracy is critical

References

- Abraham, A. (2005). “Artificial Neural Networks”. In: *Handbook of Measuring System Design*. Ed. by Editor(s) if available. Wiley.
- Aloysius, Neena and M. Geetha (2017). “A review on deep convolutional neural networks”. In: *2017 International Conference on Communication and Signal Processing (ICCSP)*. IEEE, pp. 588–592.
- Aslan, S. et al. (2020). “Benchmarking algorithms for food localization and semantic segmentation”. In: *International Journal of Machine Learning and Cybernetics* 11.12, pp. 2827–2847.
- Bhatt, Dhruv et al. (2021). “CNN variants for computer vision: History, architecture, application, challenges and future scope”. In: *Electronics* 10.20, p. 2470.
- Boykov, Yuri Y. and M.-P. Jolly (2001). “Interactive Graph Cuts for Optimal Boundary & Region Segmentation of Objects in N-D Images”. In: *Proceedings of the Eighth IEEE International Conference on Computer Vision (ICCV 2001)*. Vol. 1. IEEE, pp. 105–112.
- Chen, Liang-Chieh et al. (2017). “DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Curry, David (2025). *MyFitnessPal Revenue and Usage Statistics (2025)*. <https://www.businessofapps.com/data/myfitnesspal-statistics/>. Accessed: 2025-04-25.
- Fradkov, Alexander L. (2020). “Early History of Machine Learning”. In: *IFAC-PapersOnLine*.
- Grossi, Enzo and Massimo Buscema (2007). “Introduction to artificial neural networks”. In: *European Journal of Gastroenterology & Hepatology* 19.12, pp. 1046–1054.
- Guo, Jiuxiang et al. (2018). “Recent advances in convolutional neural networks”. In: *Pattern Recognition* 77, pp. 354–377.
- Guo, Yanbo et al. (2018). “A review of semantic segmentation using deep neural networks”. In: *International Journal of Multimedia Information Retrieval* 7, pp. 87–93.
- Haenlein, Michael and Andreas Kaplan (2019). “A Brief History of Artificial Intelligence: On the Past, Present, and Future of Artificial Intelligence”. In: *California Management Review*.
- He, Kaiming, Georgia Gkioxari, et al. (2017). “Mask R-CNN”. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2961–2969.
- He, Kaiming, Xiangyu Zhang, et al. (2016). “Deep Residual Learning for Image Recognition”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.
- Karabay, Aykut, Hasan A. Varol, and Mei Yan Chan (2025). “Improved food image recognition by leveraging deep learning and data-driven methods with an application to Central Asian Food Scene”. In: *Scientific Reports* 15, p. 14043.
- Kattenborn, Tim et al. (2021). “Review on Convolutional Neural Networks (CNN) in vegetation remote sensing”. In: *ISPRS Journal of Photogrammetry and Remote Sensing* 173, pp. 24–49.
- Kontogianni, Theodora et al. (2020). “Continuous Adaptation for Interactive Object Segmentation by Learning from Corrections”. In: *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer, pp. 1245–1262.

- Li, Jing et al. (2016). “Brief Introduction of Back Propagation (BP) Neural Network Algorithm and Its Improvement”. In: *Journal of Applied Science and Engineering* 19.3, pp. 229–239.
- Li, Yanyu et al. (2022). “EfficientFormer: Vision Transformers at MobileNet Speed”. In: *Advances in Neural Information Processing Systems*. Vol. 35, pp. 12934–12949.
- Lin, Tsung-Yi et al. (2017). “Focal Loss for Dense Object Detection”. In: *Proceedings of the IEEE International Conference on Computer Vision*. IEEE, pp. 2980–2988. DOI: 10.1109/ICCV.2017.324.
- Lo, Frank Po Wen et al. (2021). “Image-Based Food Classification and Volume Estimation for Dietary Assessment: A Review”. In: *IEEE Access* 9, pp. 91258–91275.
- Long, Jonathan, Evan Shelhamer, and Trevor Darrell (2015). “Fully convolutional networks for semantic segmentation”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3431–3440.
- Macukow, Bohdan (2016). “Neural Networks – State of Art, Brief History, Basic Models and Architecture”. In: *Proceedings of the 15th IFIP International Conference on Computer Information Systems and Industrial Management (CISIM)*. Vilnius, Lithuania: Springer, pp. 3–14. DOI: 10.1007/978-3-319-45378-1_1.
- Manning, Christopher (2022). *Artificial Intelligence Definitions*. Version 1.2, April 2022, Stanford Human-Centered Artificial Intelligence. URL: https://hai.stanford.edu/sites/default/files/2022-06/HAI_Terms_6_22.pdf.
- Minsky, Marvin and Seymour A. Papert (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- Mo, Yuting et al. (2022). “Review the state-of-the-art technologies of semantic segmentation based on deep learning”. In: *Neurocomputing* 493, pp. 626–646.
- Müller, D., I. Soto-Rey, and F. Kramer (2022). “Towards a guideline for evaluation metrics in medical image segmentation”. In: *BMC Research Notes* 15.1, p. 210. DOI: 10.1186/s13104-022-05947-2.
- Nwankpa, Chigozie et al. (2018). “Activation functions: Comparison of trends in practice and research for deep learning”. In: *arXiv preprint arXiv: 1811. 03378*.
- O’Shea, Keiron and Ryan Nash (2015). “An introduction to convolutional neural networks”. In: *arXiv preprint arXiv:1511.08458*.
- Papers With Code (2025). *State-of-the-Art Semantic Segmentation on FoodSeg103*. <https://paperswithcode.com/sota/semantic-segmentation-on-foodseg103>. Accessed: 2025-04-18.
- Popescu, Marius-Constantin et al. (2012). “Multilayer Perceptron and Neural Networks”. In: *International Journal of Advanced Computer Science and Applications* 3.3, pp. 19–24.
- Redmon, Joseph et al. (2016). “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 779–788. DOI: 10.1109/CVPR.2016.91.
- Ronneberger, Olaf, Philipp Fischer, and Thomas Brox (2015). “U-net: Convolutional networks for biomedical image segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5–9, 2015, Proceedings, Part III*. Vol. 18. Springer International Publishing, pp. 234–241.

- Rouhafzay, Asal, Ghazal Rouhafzay, and Jalila Jbilou (2025). “Image-based food monitoring and dietary management for patients living with diabetes: a scoping review of calorie counting applications”. In: *Frontiers in Nutrition* 12. Published 27 March 2025, p. 1501946. DOI: 10.3389/fnut.2025.1501946. URL: <https://doi.org/10.3389/fnut.2025.1501946>.
- Rumelhart, D. E. et al. (2013). “Backpropagation: The Basic Theory”. In: *Backpropagation*. Psychology Press, pp. 1–34.
- Siddique, Naveed et al. (2021). “U-net and its variants for medical image segmentation: A review of theory and applications”. In: *IEEE Access* 9, pp. 82031–82057.
- Simonyan, Karen and Andrew Zisserman (2014). “Very deep convolutional networks for large-scale image recognition”. In: *arXiv preprint arXiv:1409.1556*. 1556.
- Taha, Abdel Aziz A. and Allan Hanbury (2015). “Metrics for evaluating 3D medical image segmentation: analysis, selection, and tool”. In: *BMC Medical Imaging* 15.1, p. 29. DOI: 10.1186/s12880-015-0068-x.
- Taud, H. and J. F. Mas (2018). “Multilayer Perceptron (MLP)”. In: *Geomatic Approaches for Modeling Land Change Scenarios*. Springer, pp. 451–455. DOI: 10.1007/978-3-319-60801-3_26.
- Walczak, S. (2018). “Artificial Neural Networks”. In: *Encyclopedia of Information Science and Technology, Fourth Edition*. IGI Global Scientific Publishing, pp. 120–131. DOI: 10.4018/978-1-5225-2255-3.ch011.
- Walczak, Steven and Narciso Cerpa (2006). “Artificial Neural Networks”. In: *International Journal of Computer Applications* 5.2, pp. 44–49.
- Wu, Xiongwei et al. (2021). “A Large-Scale Benchmark for Food Image Segmentation”. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 11734–11743.
- Xiao, Z., Y. Li, and Z. Deng (2025). “Food image segmentation based on deep and shallow dual-branch network”. In: *Multimedia Systems* 31, p. 85.
- Xu, Ning et al. (2016). “Deep Interactive Object Selection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, pp. 373–381.
- Yamashita, Riku et al. (2018). “Convolutional neural networks: an overview and application in radiology”. In: *Insights into Imaging* 9, pp. 611–629.
- Zafar, W. et al. (2024). “Enhanced TumorNet: Leveraging YOLOv8s and U-net for superior brain tumor detection and segmentation utilizing MRI scans”. In: *Results in Engineering* 24, p. 102994.
- Zhang, Kai et al. (2019). “AIM 2019 Challenge on Constrained Super-Resolution: Methods and Results”. In: *arXiv preprint arXiv:1911.01249*. DOI: 10.48550/arXiv.1911.01249.
- Zhou, Zongwei et al. (2018). “UNet++: A Nested U-Net Architecture for Medical Image Segmentation”. In: *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support: 4th International Workshop, DLMIA 2018, and 8th International Workshop, ML-CDS 2018, held in conjunction with MICCAI 2018, Granada, Spain, September 20, 2018, Proceedings (LNCS 11045)*. Springer International Publishing, pp. 3–11.